Cmod A7-35T RISC-V MCU

Datasheet & Reference Manual

MicroBlaze-V soft MCU on the Digilent Cmod A7-35T — peripherals, pins, power, and boot guides

Platform	Xilinx Artix-7 (xc7a35tcpg236-1) — Cmod A7-35T
Processor	MicroBlaze RISC-V
Toolchain	Vivado & Vitis 2025.2

Document DS-CMOD7-0001 · Revision A · July 2026

Table of Contents

1 Device Overview

1.1 Introduction

The Cmod A7-35T RISC-V MCU is a soft microcontroller implemented on the Digilent Cmod A7-35T module. It combines a MicroBlaze-V (RISC-V) processor with tightly-coupled memory, cached external SRAM, and QSPI flash, and provides memory-mapped peripherals: GPIO, timers and PWM, UART, I2C, SPI, and an ADC.

1.2 Features

Feature	Description
Core	32-bit RISC-V (RV32IMB), 100 MHz
Cache	16 KB instruction, 16 KB data
Tightly-coupled memory	128 KB: 32 KB bootloader, 32 KB ITCM, 64 KB DTCM
External RAM	512 KB SRAM, cached
Flash	4 MB QSPI
GPIO	28 pins, 4 external interrupts
Timers / PWM	3 × 32-bit timers with interrupts, 3 PWM channels
UART	2 × 16550 UART
I2C	Master, 100 kHz
SPI	Master, ≈1.6 – 25 MHz, 2 slave selects
ADC	12-bit, 2 external inputs
I/O	48-pin DIP, 3.3 V
Boot	Standalone from flash, or JTAG from Vitis
Debug	JTAG (IEEE 1149.1)

1.3 On-Board Components

The MCU is implemented on the module's FPGA; the remaining devices provide storage, memory, communication, and power.

Component	Part	Role
FPGA	Xilinx Artix-7 (`xc7a35tcpg236-1`)	Implements the MCU: processor, memories, and peripherals
QSPI flash	Macronix MX25L3273F (Micron N25Q032A on older boards)	4 MB non-volatile storage for the hardware image and the application
SRAM	ISSI IS61WV5128BLL-10BLI	512 KB application memory
USB bridge	FTDI FT2232HQ	JTAG and UART over the Micro-USB connector
Regulator	Linear Technology LTC3569	Generates the on-board supply rails

1.4 System Block Diagram

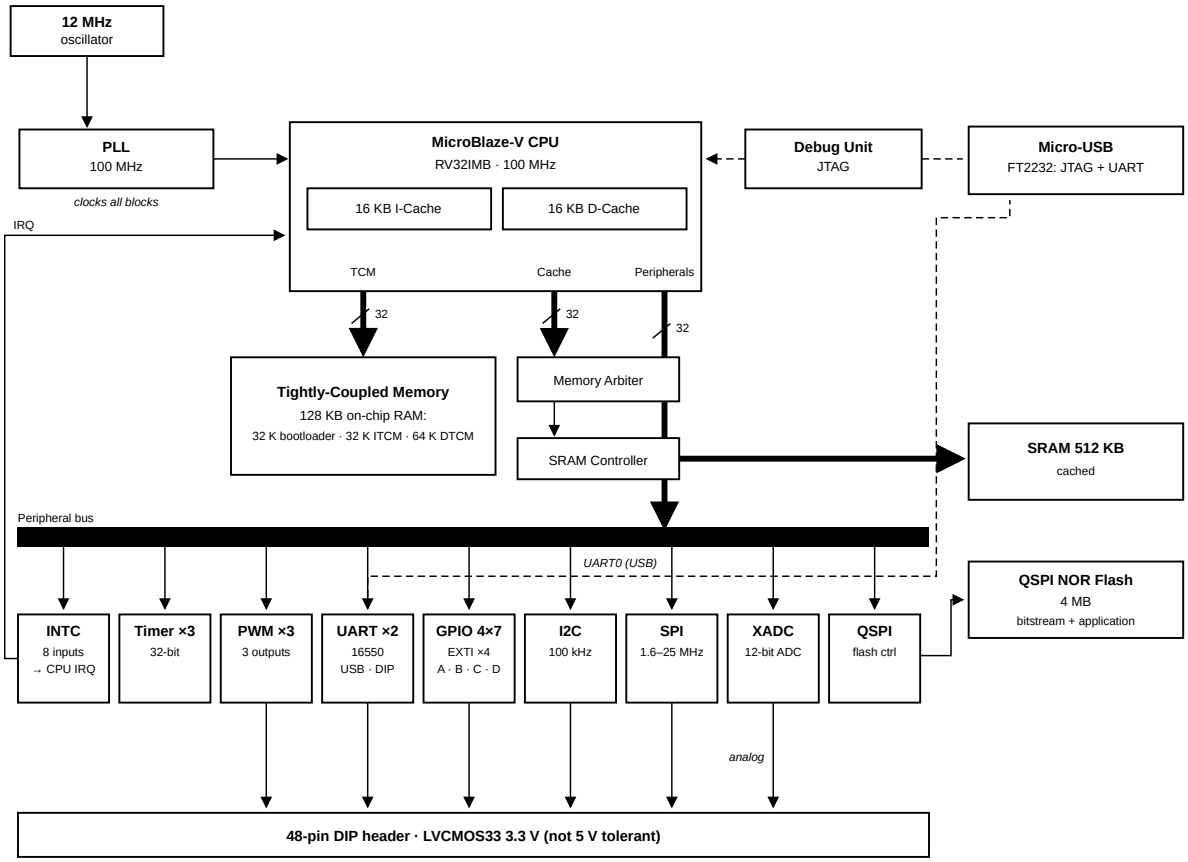


Figure 1: System Architecture

2 Pinout

2.1 DIP Connector Overview

The Cmod A7-35T has a 48-pin DIP connector. All digital I/O pins operate at LVCMOS33 (3.3 V logic level) with no series resistors.

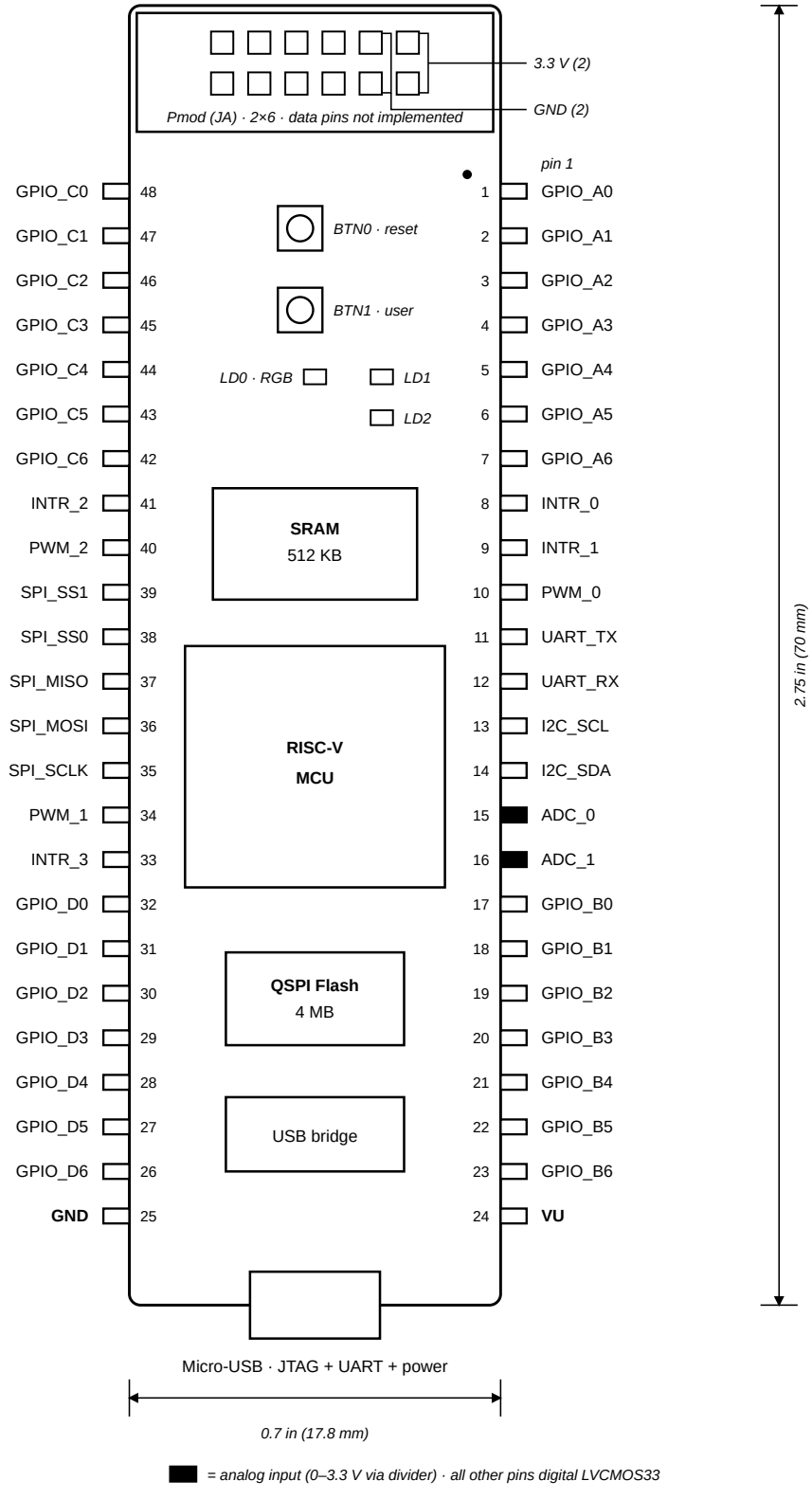


Figure 2: Cmod A7-35T DIP pinout (top view)

Peripheral registers and base addresses are listed in Section 4.6 and detailed in Section 6.

2.2 Pin Map — Left Side (Pin 1–24)

DIP Pin	Signal Name	FPGA Pin	Direction	Function
1	GPIO_A0	M3	I/O	General purpose GPIO, Group A bit 0
2	GPIO_A1	L3	I/O	General purpose GPIO, Group A bit 1
3	GPIO_A2	A16	I/O	General purpose GPIO, Group A bit 2
4	GPIO_A3	K3	I/O	General purpose GPIO, Group A bit 3
5	GPIO_A4	C15	I/O	General purpose GPIO, Group A bit 4
6	GPIO_A5	H1	I/O	General purpose GPIO, Group A bit 5
7	GPIO_A6	A15	I/O	General purpose GPIO, Group A bit 6
8	INTR_0	B15	Input	External interrupt input 0
9	INTR_1	A14	Input	External interrupt input 1
10	PWM_0	J3	Output	PWM output channel 0
11	UART_TX	J1	Output	UART 1 transmit
12	UART_RX	K2	Input	UART 1 receive
13	I2C_SCL	L1	I/O (open-drain)	I2C clock (external 4.7 kΩ pull-up recommended)
14	I2C_SDA	L2	I/O (open-drain)	I2C data (external 4.7 kΩ pull-up recommended)
15	ADC_0	G3 / G2	Analog	Analog input 0 (XADC VAUX4)
16	ADC_1	H2 / J2	Analog	Analog input 1 (XADC VAUX12)
17	GPIO_B0	M1	I/O	General purpose GPIO, Group B bit 0
18	GPIO_B1	N3	I/O	General purpose GPIO, Group B bit 1
19	GPIO_B2	P3	I/O	General purpose GPIO, Group B bit 2
20	GPIO_B3	M2	I/O	General purpose GPIO, Group B bit 3
21	GPIO_B4	N1	I/O	General purpose GPIO, Group B bit 4
22	GPIO_B5	N2	I/O	General purpose GPIO, Group B bit 5
23	GPIO_B6	P1	I/O	General purpose GPIO, Group B bit 6
24	**VU**	—	Power	Power input (ext) / Power output (USB)

2.3 Pin Map — Right Side (Pin 25–48)

DIP Pin	Signal Name	FPGA Pin	Direction	Function
25	**GND**	—	Power	Ground reference
26	GPIO_D6	R3	I/O	General purpose GPIO, Group D bit 6
27	GPIO_D5	T3	I/O	General purpose GPIO, Group D bit 5
28	GPIO_D4	R2	I/O	General purpose GPIO, Group D bit 4
29	GPIO_D3	T1	I/O	General purpose GPIO, Group D bit 3
30	GPIO_D2	T2	I/O	General purpose GPIO, Group D bit 2
31	GPIO_D1	U1	I/O	General purpose GPIO, Group D bit 1
32	GPIO_D0	W2	I/O	General purpose GPIO, Group D bit 0
33	INTR_3	V2	Input	External interrupt input 3
34	PWM_1	W3	Output	PWM output channel 1
35	SPI_SCLK	V3	Output	SPI clock (software-settable, up to 25 MHz)
36	SPI_MOSI	W5	Output	SPI master-out slave-in
37	SPI_MISO	V4	Input	SPI master-in slave-out
38	SPI_SS0	U4	Output	SPI slave select 0 (active low)
39	SPI_SS1	V5	Output	SPI slave select 1 (active low)
40	PWM_2	W4	Output	PWM output channel 2
41	INTR_2	U5	Input	External interrupt input 2
42	GPIO_C6	U2	I/O	General purpose GPIO, Group C bit 6
43	GPIO_C5	W6	I/O	General purpose GPIO, Group C bit 5
44	GPIO_C4	U3	I/O	General purpose GPIO, Group C bit 4

DIP Pin	Signal Name	FPGA Pin	Direction	Function
45	GPIO_C3	U7	I/O	General purpose GPIO, Group C bit 3
46	GPIO_C2	W7	I/O	General purpose GPIO, Group C bit 2
47	GPIO_C1	U8	I/O	General purpose GPIO, Group C bit 1
48	GPIO_C0	V8	I/O	General purpose GPIO, Group C bit 0

2.4 On-Board I/O (No DIP Pin Exposure)

These devices are mounted on the module and use no DIP pins; the names match the board silkscreen. The LEDs and the user button are GPIO ports; the reset button is wired to the system reset.

Device	Function
LD0	RGB LED (three GPIO outputs)
LD1, LD2	Status LEDs (GPIO outputs)
BTN1	User button (GPIO input)
BTN0	Reset button (not readable by software)

3 Electrical Characteristics and Power

3.1 Electrical Characteristics

Recommended Operating Conditions

Parameter	Value
I/O logic level	LVCMOS33 (3.3 V)
Digital input voltage	0 V to 3.3 V
Analog input voltage (pins 15, 16)	0 V to 3.3 V

Absolute Maximum Ratings

Stresses beyond these ratings may cause permanent damage to the device.

Parameter	Min	Max
Digital I/O pin voltage	–0.4 V	3.85 V

> **Not 5 V tolerant:** The DIP pins connect to the device with no series resistors. Drive them only with 3.3 V logic; use a level shifter or resistive divider for any 5 V signal.

> **Warning:** Disconnect any external supply from the VU pin before connecting USB. When a USB host is attached, VU is driven to the USB voltage (4.5–5.5 V), which can damage a source feeding VU — a battery especially, since it cannot tolerate reverse current.

3.2 Power Architecture

The Cmod A7-35T uses a Linear Technology LTC3569 triple-output buck regulator to generate all onboard supply voltages from a single input rail called VU.

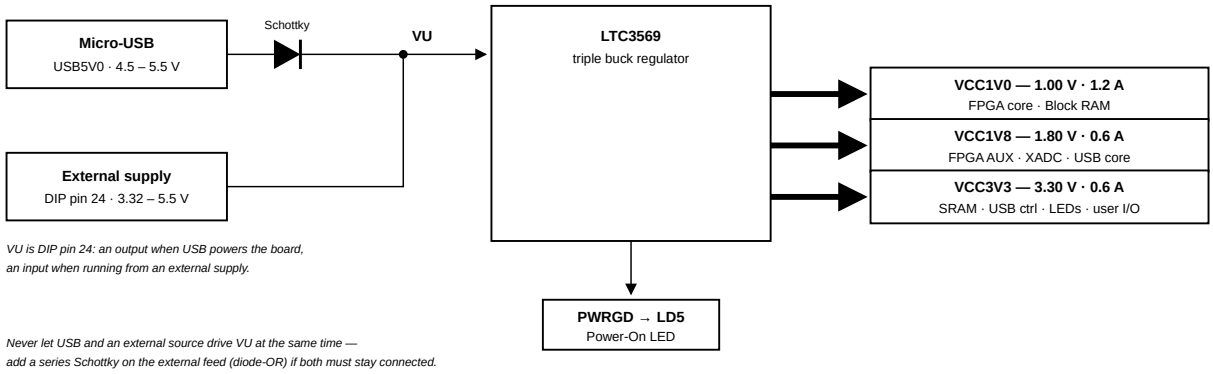


Figure 3: Cmod A7 power supply tree

3.3 Power Input Options

The board can be powered from either of the following sources:

Source	Connector	Schematic Net	Min Voltage	Recommended	Max Voltage
USB	Micro-USB	USB5V0	4.5 V	5.0 V	5.5 V
External Supply	DIP pin 24	VU	3.32 V*	5.0 V	5.5 V

> **Note:** * The minimum external voltage on VU depends on the current drawn from the VCC3V3 rail via the Pmod header:
>
> - 0 mA drawn: minimum 3.32 V
> - 100 mA drawn: minimum 3.38 V
> - 250 mA drawn: minimum 3.48 V

The GND reference is DIP pin 25.

3.4 Output Power Rails

All three rails are generated by the LTC3569 from the VU input.

Rail	Typical Voltage	Min / Max Voltage	Max Current	Powered Devices
VCC1V0	1.00 V	0.95 V / 1.05 V	1.2 A	FPGA Core, Block RAM
VCC1V8	1.80 V	1.71 V / 1.89 V	0.6 A	FPGA AUX, ADC, USB Core
VCC3V3	3.30 V	3.135 V / 3.465 V	0.6 A	SRAM, USB Controller, Pmod, LEDs, Buttons, FPGA User I/O

3.4.1 Powering External Circuits

The Pmod connector is the board's 3.3 V supply point for user circuits: its two VCC pins provide regulated 3.3 V and its two GND pins the return. The DIP header exposes only VU and GND, so an external 3.3 V module is powered from the Pmod and its signals are routed through the DIP GPIO pins. The Pmod data pins are not implemented in this device.

These pins share the 0.6 A VCC3V3 rail with the on-board devices; reserve them for low-current loads such as sensor modules, and give motors their own supply. When the board runs from an external supply, the minimum VU voltage rises with the current drawn here (see the note in Section 3.3).

3.5 VU Pin Behavior

Condition	VU Pin Function
USB connected	Output. Driven from USB 5 V through a Schottky diode; approximately 5 V after the diode drop
USB not connected	Input. Accepts an external supply of 3.32–5.5 V

3.6 Dual Power Source (USB + External)

An on-board Schottky diode isolates the USB port from the VU node; the VU pin itself has no diode. The behavior of each configuration is as follows:

Configuration	Behavior
USB only	Normal operation
External supply only	Normal operation
Both connected, one source active	Normal operation. The inactive source supplies no current.
Both sources active	Not permitted. The two supplies drive the VU node against each other.
Both sources active, with a series Schottky on the external feed	Permitted. The higher voltage supplies the board; the other is blocked (diode-OR).

> **Note:** When both sources must operate at the same time (for example, USB for the console while powering from an external supply), add a Schottky diode in series with the external supply at the VU pin.

4 System Architecture

4.1 MicroBlaze RISC-V Core

Item	Value
Architecture	32-bit RISC-V, RV32IMB (hardware multiply/divide, bit-manipulation)
Clock	100 MHz
Cache	16 KB instruction + 16 KB data, write-through, 32-byte lines
Cached region	External SRAM only
Debug	JTAG: breakpoints, register and memory access

Description: The processor executes firmware from SRAM. All memory and peripherals are memory-mapped into one address space. The external SRAM is cached; the tightly-coupled memory and peripheral registers are not, so peripheral reads and writes always take effect immediately.

4.2 Clock

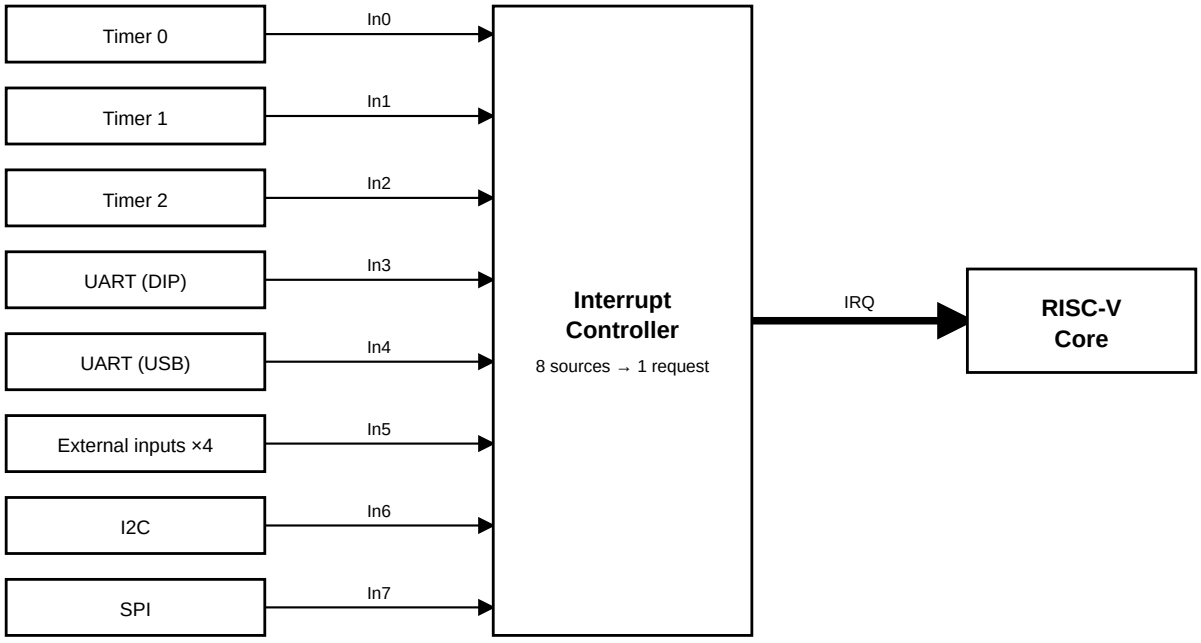
Description: The system clock is 100 MHz, generated from the 12 MHz on-board oscillator by an on-chip PLL. The processor and all peripherals run at this frequency.

4.3 Reset

Description: A reset restarts the processor from the bootloader, which reloads the application from flash (see Section 8), and returns peripheral registers to their reset values. There are three reset sources: power-on, the on-board reset button BTN0, and a system reset issued by the debugger.

4.4 Interrupt Controller

The interrupt controller combines the peripheral interrupt sources into a single request line to the processor. When an interrupt is taken, the processor reads the controller to determine which source is active.



A source reaches the core only when the source, the controller, and the core all enable interrupts.

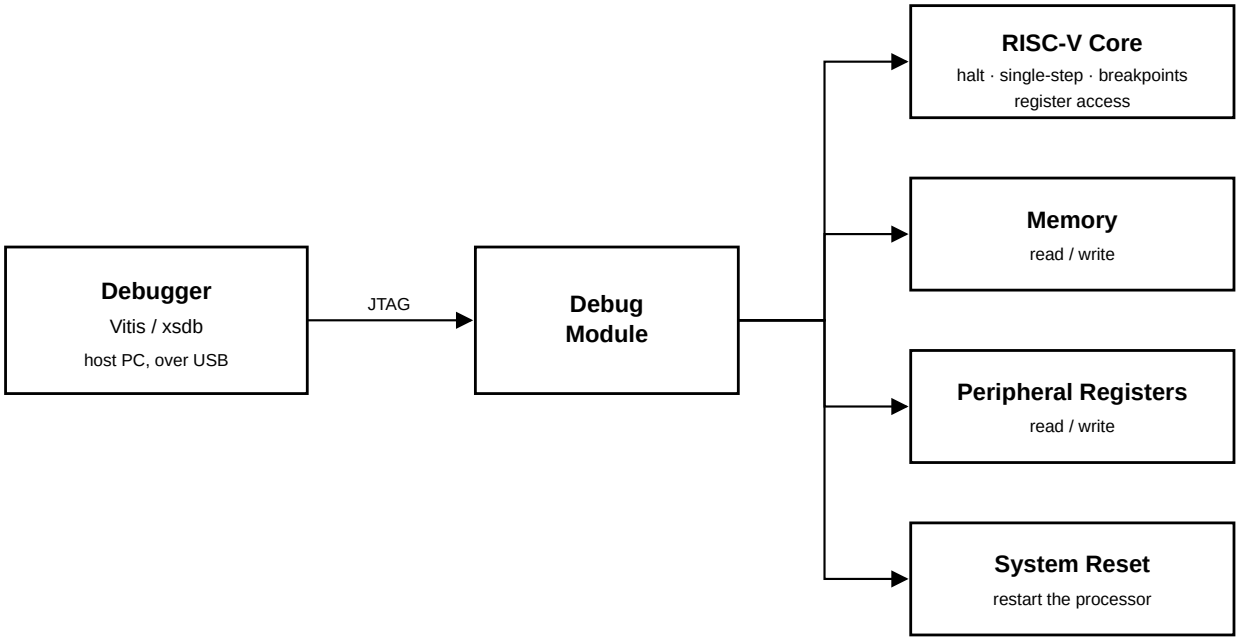
Figure 4: The interrupt controller collects eight sources into one request to the core

Interrupt Mapping:

Channel	Source	Description
In0	`timer_0`	System timer 0 interrupt
In1	`timer_1`	System timer 1 interrupt
In2	`timer_2`	System timer 2 interrupt
In3	`uart_1`	External UART interrupt
In4	`uart_USB`	USB UART interrupt
In5	`INT_0_3`	External GPIO interrupt (4-bit)
In6	`i2c_0`	I2C controller interrupt
In7	`spi_0`	External SPI master interrupt

4.5 Debug Module

Description: The debug module connects a host debugger to the processor over the JTAG port. Vitis and `xsdb` sessions use it to halt and single-step the core, set breakpoints, read and write the processor registers, memory, and peripheral registers, and reset the system. Because every peripheral is memory-mapped, the debugger can operate a device directly — for example writing a GPIO register to drive an output — with no firmware running.



Debug access is implemented in hardware; the core can be halted even when the application is unresponsive.

Figure 5: The debug module gives the host debugger access to the core, memory, peripheral registers, and system reset over JTAG

4.6 Complete Address Map

Peripheral addresses follow a class-based convention: `0x40[C]x_xxxx`, where `C` is the peripheral class (1 MB per class, 64 KB per instance). The device type is identifiable directly from the address: class 0 = GPIO, 1 = Timer, 2 = PWM, 3 = UART, 4 = INTC, 5 = QSPI control, 6 = XADC, 7 = I2C, 8 = SPI, 9 = SPI clock control.

Base Address	Range	Peripheral	Category
`0x0000_0000`	128K	Tightly-coupled memory (bootloader, ITCM, DTCM)	Memory
`0x4000_0000`	64K	On-board LEDs	GPIO
`0x4001_0000`	64K	User button (BTN1)	GPIO
`0x4002_0000`	64K	RGB LED	GPIO
`0x4003_0000`	64K	GPIO group A	GPIO
`0x4004_0000`	64K	GPIO group B	GPIO
`0x4005_0000`	64K	GPIO group C	GPIO
`0x4006_0000`	64K	GPIO group D	GPIO
`0x4007_0000`	64K	External interrupt inputs	Interrupt
`0x4010_0000`	64K	Timer 0	Timer
`0x4011_0000`	64K	Timer 1	Timer
`0x4012_0000`	64K	Timer 2	Timer
`0x4020_0000`	64K	PWM 0	PWM
`0x4021_0000`	64K	PWM 1	PWM
`0x4022_0000`	64K	PWM 2	PWM
`0x4030_0000`	64K	USB UART	Communication
`0x4031_0000`	64K	External UART (DIP 11/12)	Communication
`0x4040_0000`	64K	Interrupt controller	System
`0x4050_0000`	64K	QSPI flash controller	Memory
`0x4060_0000`	64K	ADC (XADC)	ADC
`0x4070_0000`	64K	I2C controller (DIP 13/14)	Communication
`0x4080_0000`	64K	SPI master (DIP 35–39)	Communication
`0x4090_0000`	64K	SPI clock control	Communication
`0x6000_0000`	512K	External SRAM (cached)	Memory

5 Memory Hierarchy

5.1 Memory Hierarchy Overview

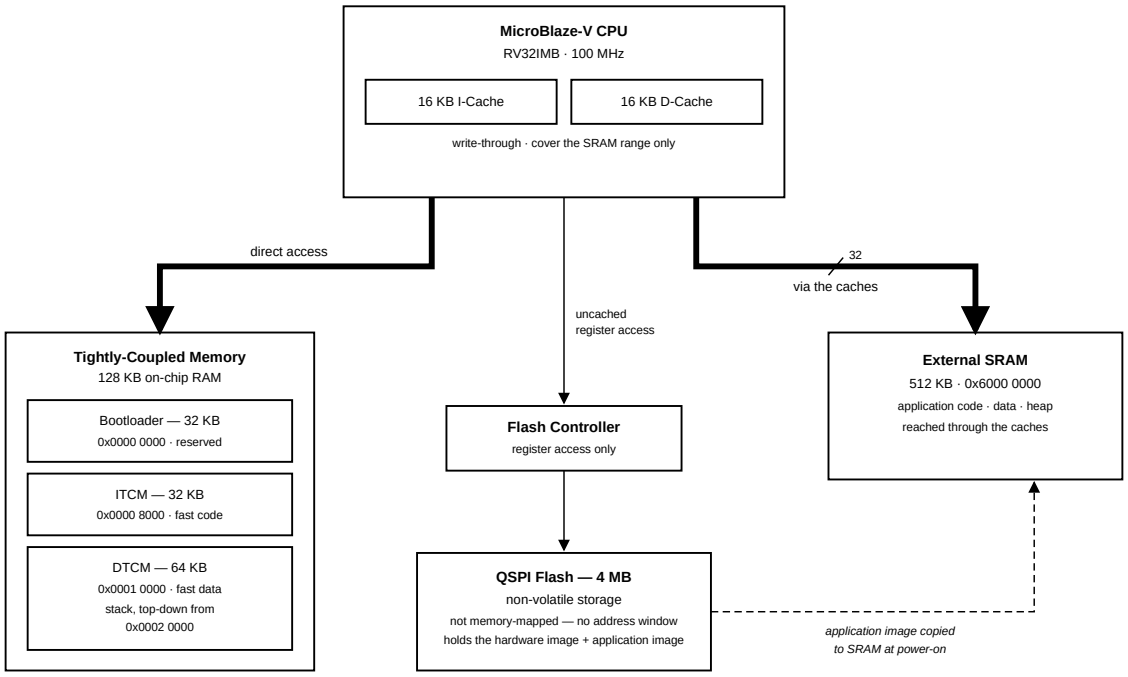


Figure 6: Memory Hierarchy

Memory	Size	Role
Tightly-coupled memory	128 KB	Interrupt handlers, stack, frequently accessed data
I-Cache + D-Cache	16 KB + 16 KB	Transparent acceleration of SRAM
External SRAM	512 KB	Application code, data, heap
QSPI flash	4 MB	Non-volatile: application + hardware image

5.2 Memory Map

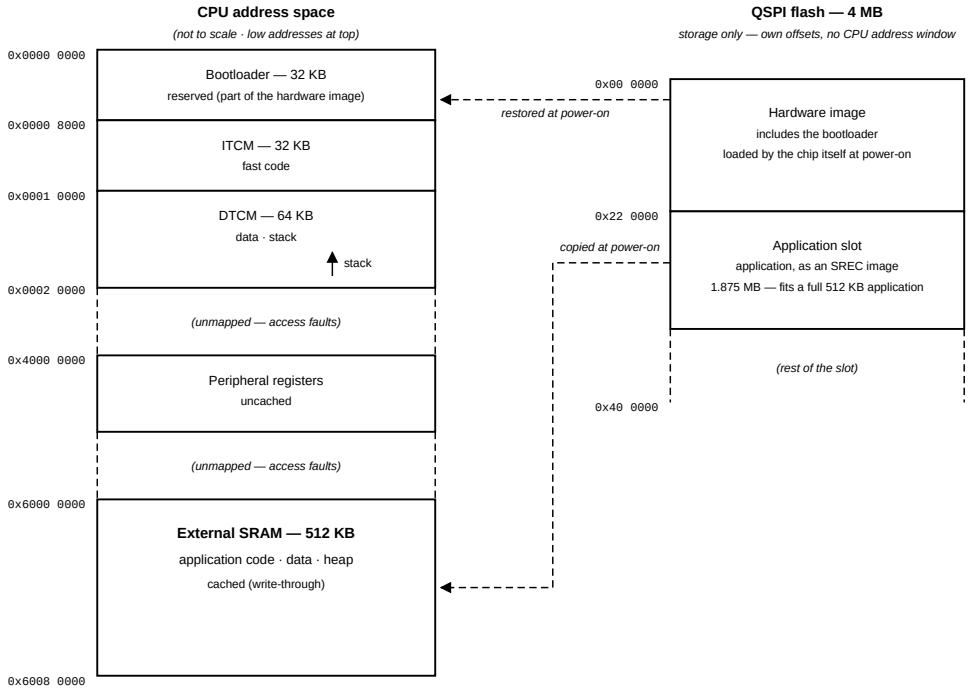


Figure 7: Memory Map

Address range	Size	Region	Access
`0x0000_0000` – `0x0000_7FFF`	32 KB	Bootloader (reserved)	uncached
`0x0000_8000` – `0x0000_FFFF`	32 KB	ITCM — application code	uncached
`0x0001_0000` – `0x0001_FFFF`	64 KB	DTCM — data; stack grows down from `0x0002_0000`	uncached
`0x4000_0000` – `0x40FF_FFFF`	—	Peripheral registers	uncached
`0x6000_0000` – `0x6007_FFFF`	512 KB	External SRAM — application code / data / heap	cached

5.3 Tightly-Coupled Memory (ITCM / DTCM / Bootloader)

Item	Value
Address	`0x0000_0000` – `0x0001_FFFF` (128 KB)
Ports	Dual-port: simultaneous instruction and data access
Layout	32 KB bootloader + 32 KB ITCM + 64 KB DTCM

Description: The tightly-coupled memory (TCM) is the on-chip RAM, accessed outside the cache path. It is a dual-port memory, so an instruction fetch and a data access can proceed at the same time. It serves the same role as ITCM/DTCM on other MCU cores (e.g. Cortex-M7), and is divided into the bootloader region, the ITCM for interrupt handlers and timing-critical code, and the DTCM for the stack and fast data.

> **Note:** The bootloader region has no software load path. Its contents are carried inside the hardware image and restored from flash at every power-on, before the processor starts. Like a mask ROM, the bootloader survives application updates and cannot be corrupted by software (see Section 8).

5.4 External SRAM

Item	Value
Base Address	`0x6000_0000`
Size	512 KB — `0x6000_0000` – `0x6007_FFFF`
Memory	On-board asynchronous SRAM
Cache	Covered by the I-Cache and D-Cache

Description: This memory controller interfaces to the on-board 512 KB SRAM, the main application memory: standalone boot copies the program here and executes it.

5.5 QSPI Flash

Item	Value
Base Address	`0x4050_0000`
Flash Device	4 MB QSPI NOR flash: Macronix MX25L3273F, or Micron N25Q032A on older boards
Mode	CPU-programmable register mode — flash contents are not memory-mapped
FIFO	256 B TX/RX — one full flash page (256 B) per transfer

Description: This controller manages the on-board Quad-SPI NOR Flash. The CPU drives the flash through the controller, so firmware can read, erase, and program it at runtime.

Flash contents are not memory-mapped: there is no XIP window, and code cannot execute from flash directly. The standalone-boot design instead copies the application from flash into SRAM at power-on (see Section 8).

> **Design note:** A read-only memory-mapped XIP window and CPU-programmable register mode are mutually exclusive in this controller. This MCU uses register mode: keeping the flash CPU-programmable at runtime is preferred over execute-in-place, and execution is served by the 512 KB SRAM and the I-cache instead.

5.6 Caches

Item	Value
Instruction cache	16 KB, 32-byte lines
Data cache	16 KB, 32-byte lines
Write policy	write-through — every store is forwarded to SRAM
Cached range	exactly the SRAM: `0x6000_0000` – `0x6007_FFFF`

Only the SRAM range is cached. The tightly-coupled memories are already fast enough, and peripheral registers must observe every access, so neither is ever cached. The caches are always on for the cached range and require no software management.

The write-through policy has two practical consequences:

- Reads benefit fully from the cache: a hit has the same latency as a TCM access.
- Writes always incur the SRAM latency, even on a cache hit. Frequently written data (counters, buffers filled in interrupt handlers) should therefore be placed in the DTCM.

5.7 Measured Access Latencies

The figures below were measured on the board at 100 MHz with an optimized (`-O2`) word-copy loop and a hardware cycle counter; the cache-miss figure was taken immediately after invalidating the data cache. Each value is the latency of a single access.

Access	Cycles	Notes
TCM read	1	single-cycle; identical every time
SRAM read, cache **hit**	1	a hit matches the TCM
SRAM read, cache **miss**	≈151	fetches one 32-byte line (≈19 cycles per word)
SRAM write (write-through)	≈16	per store; the store waits for SRAM

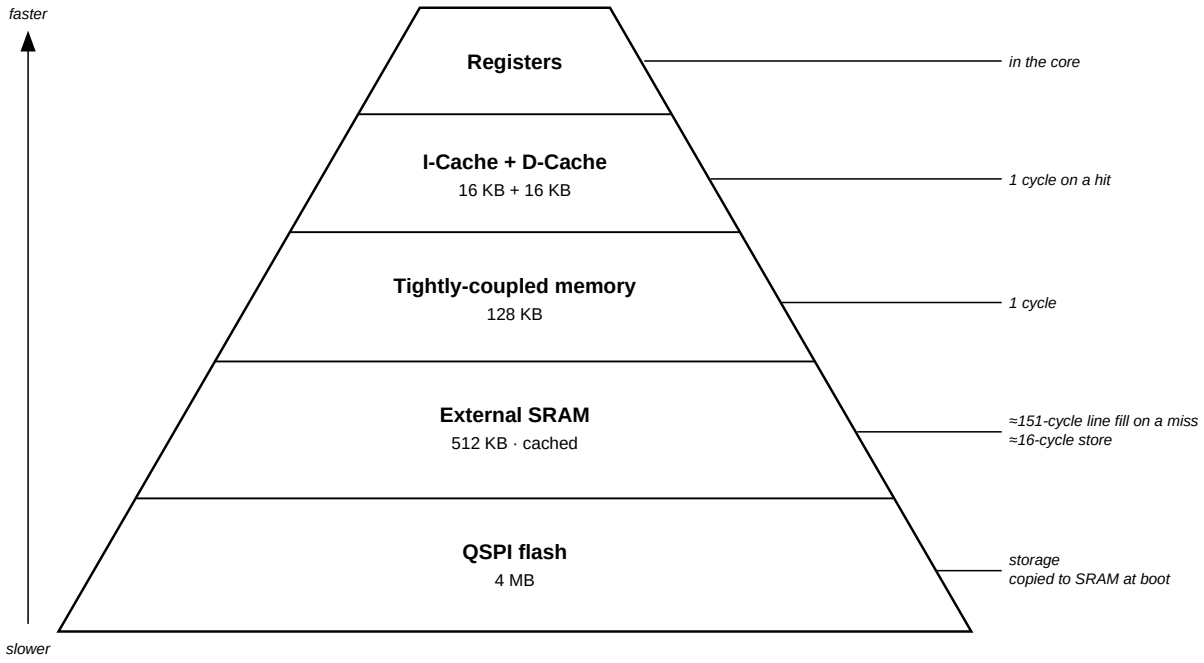


Figure 8: Memory hierarchy: faster and smaller toward the top, larger and slower toward flash

5.8 Memory Placement Guidance

What	Put it in
Interrupt handlers, timing-critical loops	ITCM
Frequently-written data, buffers	DTCM
Stack	DTCM
Everything else — code, constants, data, heap	SRAM
Firmware image	QSPI flash

As a general rule, place code and data in SRAM by default and move them to the TCMs when predictable timing is required.

A function in SRAM usually runs at cache speed, but its worst case (a cold cache line) takes three to four times longer; for an interrupt handler this spread appears directly as latency jitter.

In the tightly-coupled memories the worst-case latency equals the best-case latency.

6 Peripherals

All peripheral registers are memory-mapped; base addresses are listed in Section 4.6. Pin locations for every signal are listed in the pin maps of Section 2. The QSPI flash controller is described with the memory system in Section 5.5.

6.1 Register Access Conventions

Every peripheral register in this device is a 32-bit word at a word-aligned offset from its instance base address (the Complete Address Map lists all base addresses). Registers must be accessed with full-word reads and writes: in C through `Xil_In32`/`Xil_Out32` or a `volatile` pointer, in assembly with `lw`/`sw`. Byte and halfword accesses are not supported by the peripheral bus.

The register tables in this chapter use the following access codes:

Code	Meaning
RW	Read/write
RO	Read-only; writes are ignored
WO	Write-only; reads return undefined data
R/W1C	Readable; writing 1 to a set bit clears it
R/TOW	Readable; writing 1 toggles the bit (see the GPIO `ISR`)
RSE	Read side effect: the read itself changes state (for example, a UART receive-buffer read pops the FIFO)

Reserved bits read as undefined values and must be written as 0. Reading a register with a read side effect purely for inspection (for example from a debugger memory view) consumes data or clears status.

Reset values apply after power-on or reset.

> **Note:** Loading a new program over JTAG does not reset the peripherals: registers keep whatever state the previous program left, including armed interrupts and half-finished bus transactions. Programs must not assume reset values at entry; initialize every peripheral before use.

The following two fragments are equivalent and drive the RGB LED port directly — the C form is the course template's idiom, the assembly form is the `asm_template` idiom:

```c
Xil_Out32(0x40020000 + 0x4, 0x0);   /* TRI: all pins outputs */
Xil_Out32(0x40020000, 0x2);         /* DATA: green (bit 1)   */
```

```asm
li   t0, 0x40020000        # RGB LED port base
sw   zero, 4(t0)           # TRI: all pins outputs
li   t1, 0x2               # bit 1 = green
sw   t1, 0(t0)             # DATA: drive the pins
```

6.2 GPIO

All GPIO groups are memory-mapped ports with per-bit direction control: the TRI register sets each pin's direction, and the DATA register reads or writes the pin. The Vitis driver is `XGpio`.

6.2.1 On-Board GPIO

Instance	Base Address	Width	Direction	Connection	Description
`board_led_2bits`	`0x4000_0000`	2	Output	LD1, LD2	On-board LEDs × 2
`board_button`	`0x4001_0000`	1	Input	BTN1	On-board user button
`board_rgb`	`0x4002_0000`	3	Output	LD0	On-board RGB LED (bit 0 = Blue, bit 1 = Green, bit 2 = Red)

6.2.2 DIP Connector GPIO (4 Groups × 7-bit)

Instance	Base Address	Width	DIP Pins	Description
`gpio_A_0_6`	`0x4003_0000`	7	Pin 1–7	GPIO Group A, bidirectional I/O
`gpio_B_0_6`	`0x4004_0000`	7	Pin 17–23	GPIO Group B, bidirectional I/O
`gpio_C_0_6`	`0x4005_0000`	7	Pin 42–48	GPIO Group C, bidirectional I/O
`gpio_D_0_6`	`0x4006_0000`	7	Pin 26–32	GPIO Group D, bidirectional I/O

Description: Each group is a 7-bit bidirectional port; every bit can be an input or an output independently.

6.2.3 External Interrupt Inputs (`INT_0_3`)

Item	Value
Base Address	`0x4007_0000`
Width	4-bit, input-only
DIP Pins	Pin 8 (INTR_0), Pin 9 (INTR_1), Pin 41 (INTR_2), Pin 33 (INTR_3)
Interrupt	INTC In5

Description: Four external interrupt inputs are grouped into a single GPIO instance with interrupt generation enabled. A change on any of the four pins can raise INTC In5.

6.2.4 GPIO Register Map

All eight GPIO ports share the same register layout. Offsets are relative to the port base address listed in the tables above. Only the low WIDTH bits of each register are implemented (7 for groups A–D, 4 for `INT_0_3`, 3 for the RGB port, 2 for the LED port, 1 for the button port); unimplemented bits read as 0. Offsets `0x08`/`0x0C` (second-channel data and direction) are not implemented in this device, and the interrupt registers (`0x11C`–`0x128`) are implemented on the `INT_0_3` port only.

Offset	Name	Access	Reset	Description
`0x000`	`DATA`	RW	`0x0`	Port data
`0x004`	`TRI`	RW	all 1	Per-pin direction (1 = input, 0 = output)
`0x11C`	`GIER`	RW	`0x0`	Global interrupt enable (`INT_0_3` only)
`0x120`	`ISR`	R/TOW	`0x0`	Interrupt status (`INT_0_3` only)
`0x128`	`IER`	RW	`0x0`	Interrupt enable (`INT_0_3` only)

6.2.4.1 DATA — Port Data Register (Offset 0x000)

Bits	Name	Access	Description
31:W	—	—	Reserved (W = port width). Read as 0.
W-1:0	`DATA`	RW	One bit per pin. A read returns the pin level for input pins and the last written value for output pins. A write drives pins configured as outputs; writes to input pins have no effect.

6.2.4.2 TRI — Port Direction Register (Offset 0x004)

Bits	Name	Access	Description
31:W	—	—	Reserved. Read as 0.
W-1:0	`TRI`	RW	Per-pin direction: 1 = input, 0 = output. All pins are inputs after reset; software must clear the relevant bits before driving a pin. On the fixed-direction ports (LEDs, RGB, button, `INT_0_3`) the direction is set in hardware and this register has no effect.

6.2.4.3 GIER — Global Interrupt Enable Register (Offset 0x11C)

Bits	Name	Access	Description
31	`GIE`	RW	Master gate for the port's interrupt output. Write `0x8000_0000` to enable.
30:0	—	—	Reserved.

6.2.4.4 ISR — Interrupt Status Register (Offset 0x120)

Bits	Name	Access	Description
31:1	—	—	Reserved.
0	`CH1`	R/TOW	Set by hardware when any input pin of the port changes state (either edge). Toggle-on-write: to clear, read the register and write the read value back.

6.2.4.5 IER — Interrupt Enable Register (Offset 0x128)

Bits	Name	Access	Description
31:1	—	—	Reserved.
0	`CH1`	RW	Enables the port interrupt. The enable is per port, not per pin: any of the four `INT_0_3` inputs raises the same interrupt, and the handler reads `DATA` to determine which line changed.

Driver support (`XGpio`). The functions below cover the course use of the driver; the last column names the registers each one accesses.

Function	Purpose	Registers touched
`XGpio_Initialize(&inst, base)`	Bind the instance to a port	none
`XGpio_SetDataDirection(&inst, 1, m)`	Set pin directions	`TRI`
`XGpio_DiscreteRead(&inst, 1)`	Read pin states	`DATA`
`XGpio_DiscreteWrite(&inst, 1, v)`	Write the whole port	`DATA`
`XGpio_DiscreteSet/Clear(&inst, 1, m)`	Set or clear only masked bits	`DATA` (read-modify-write)
`XGpio_InterruptEnable(&inst, 1)`	Enable the port interrupt	`IER`
`XGpio_InterruptGlobalEnable(&inst)`	Open the interrupt gate	`GIER`
`XGpio_InterruptGetStatus(&inst)`	Read pending status	`ISR`
`XGpio_InterruptClear(&inst, 1)`	Acknowledge the interrupt	`ISR` (toggle-safe)

Note: Three behaviors are board-verified sources of bugs. `ISR` toggles on write: writing a hard-coded 1 while the bit is clear sets a phantom pending interrupt, so always write back exactly the value just read (or use `XGpio_InterruptClear`). `XGpio_DiscreteSet/Clear` are read-modify-write sequences, not atomic: if an interrupt handler writes the same port, keep all writes to that port in one context. Finally, `XGpio_Initialize` in this toolchain takes the port base address (`XPAR_..._BASEADDR`); older examples that pass a device ID do not compile here.

6.3 Timers and PWM

The device provides six 32-bit timer instances (Vitis driver: `XTmrCtr`): three serve as general-purpose timers with interrupts, and three have their outputs routed to pins as PWM channels.

6.3.1 System Timers

Instance	Base Address	Interrupt	Description
`timer_0`	`0x4010_0000`	INTC In0	General-purpose system timer
`timer_1`	`0x4011_0000`	INTC In1	General-purpose timer
`timer_2`	`0x4012_0000`	INTC In2	General-purpose timer

6.3.2 PWM Outputs

Instance	Base Address	Output Pin	DIP Pin	Description
`PWM_0`	`0x4020_0000`	J3	Pin 10	PWM Channel 0
`PWM_1`	`0x4021_0000`	W3	Pin 34	PWM Channel 1
`PWM_2`	`0x4022_0000`	W4	Pin 40	PWM Channel 2

Description: These timer instances are configured in PWM mode to generate square-wave outputs, suitable for applications such as LED dimming, motor speed control, and buzzer tone generation. Frequency and duty cycle are configured through the Timer Load Registers and the PWM enable bit.

6.3.3 Timer/PWM Register Map

Each of the six instances (`timer_0`–`timer_2`, `PWM_0`–`PWM_2`) is one dual 32-bit counter block: counter 0 occupies offsets `0x00`–`0x08` and counter 1 the same layout at `+0x10`. The general-purpose timers normally use counter 0 alone; PWM mode uses both counters together (period and high time). All counters run at the 100 MHz bus clock, so one count equals 10 ns.

Offset	Name	Access	Reset	Description
`0x00`	`TCSR0`	RW	`0x0`	Counter 0 control and status
`0x04`	`TLR0`	RW	`0x0`	Counter 0 load value
`0x08`	`TCR0`	RO	`0x0`	Counter 0 current count
`0x10`	`TCSR1`	RW	`0x0`	Counter 1 control and status
`0x14`	`TLR1`	RW	`0x0`	Counter 1 load value
`0x18`	`TCR1`	RO	`0x0`	Counter 1 current count

6.3.3.1 TCSR — Control and Status Register (Offsets 0x00 and 0x10)

Bits	Name	Access	Description
31:12	—	—	Reserved. Write 0.
11	`CASC`	RW	Cascade both counters into one 64-bit counter.
10	`ENALL`	RW	Writing 1 enables both counters simultaneously.

Bits	Name	Access	Description
9	PWMA	RW	PWM mode (set in both TCSRs, together with GENT).
8	TINT	R/W1C	Interrupt status: reads 1 after the counter reaches its terminal value. Write the register back with this bit set to clear it.
7	ENT	RW	Enable the counter (starts counting).
6	ENIT	RW	Enable the interrupt output (general-purpose timers only).
5	LOAD	RW	While 1, forces TCR = TLR. Write a pulse (set, then clear) to load; the counter does not run while set.
4	ARHT	RW	Auto-reload: reload TLR and continue at the terminal value (1) or stop and hold (0).
3	CAPT	RW	External capture mode. Not wired in this device; write 0.
2	GENT	RW	Enable the generate output (required for PWM).
1	UDT	RW	Count direction: 0 = up, 1 = down.
0	MDT	RW	Mode: 0 = generate, 1 = capture. Write 0.

6.3.3.2 TLR — Load Register (Offsets 0x04 and 0x14)

The 32-bit value loaded into the counter by LOAD or by auto-reload. In PWM mode TLR0 sets the period and TLR1 the high time; the hardware adds a fixed two-cycle overhead to each interval.

6.3.3.3 TCR — Counter Register (Offsets 0x08 and 0x18)

The live 32-bit count, readable at any time without disturbing the counter.

Configuration recipes (board-verified, from the course examples):

Free-running cycle counter — count up from zero and read elapsed 10 ns cycles:

```
Xil_Out32(TIMER + 0x00, 0x0);          /* TCSR0: stop, clear mode  */
Xil_Out32(TIMER + 0x04, 0x0);          /* TLR0: start value 0      */
Xil_Out32(TIMER + 0x00, 0x20);         /* LOAD pulse: TCR0 <- TLR0 */
Xil_Out32(TIMER + 0x00, 0x80);         /* ENT: run                 */
u32 cycles = Xil_In32(TIMER + 0x08);   /* TCR0: elapsed cycles     */
```

PWM output — TLR0 = period, TLR1 = high time, both counters in PWM-generate mode (0x296 = PWMA|T-run|ARHT|GENT|UDT):

```
Xil_Out32(PWM + 0x04, PERIOD_CYC);     /* TLR0: period             */
Xil_Out32(PWM + 0x00, 0x20);           /* LOAD counter 0           */
Xil_Out32(PWM + 0x14, HIGH_CYC);       /* TLR1: high time          */
Xil_Out32(PWM + 0x10, 0x20);           /* LOAD counter 1           */
Xil_Out32(PWM + 0x00, 0x296);          /* TCSR0: PWMA|ENT|ARHT|GENT|UDT */
Xil_Out32(PWM + 0x10, 0x296);          /* TCSR1: same              */
```

Driver support (`XTmrCtr`).

Function	Purpose	Registers touched
XTmrCtr_Initialize(&inst, base)	Bind the instance and stop both counters	TCSR0/1
XTmrCtr_SetOptions(&inst, n, opt)	Set mode bits (down-count, auto-reload, interrupt)	TCSR
XTmrCtr_SetResetValue(&inst, n, v)	Set the load value	TLR
XTmrCtr_Start(&inst, n)	Load and start counter *n*	TCSR (LOAD, then ENT)
XTmrCtr_Stop(&inst, n)	Stop counter *n*	TCSR
XTmrCtr_GetValue(&inst, n)	Read the live count	TCR

> **Note:** The PWM instances are ordinary timer blocks whose outputs are routed to pins — there are no separate PWM registers. In interrupt use, the handler must clear TINT by writing TCSR back with bit 8 set; an unacknowledged timer interrupt re-enters the handler forever. A program loaded over JTAG on top of a running program can inherit an already-armed timer interrupt, so interrupt-driven programs should disable and acknowledge the timers during initialization before enabling their own handlers.

6.4 UART

6.4.1 USB UART (`uart_USB`)

Item	Value
Base Address	`0x4030_0000`
TX / RX Pins	J18 / J17 — routed to the on-board Micro-USB connector
Interrupt	INTC In4

Description: A 16550-compatible UART provides host PC communication through the on-board Micro-USB connector. The baud rate is software-configured via the divisor latch: at a 100 MHz system clock, divisor 54 (0x36) gives 115200 baud. This UART is typically mapped as STDIN/STDOUT.

6.4.2 External UART (`uart_1`)

Item	Value
Base Address	`0x4031_0000`
TX / RX Pins	J1 (DIP 11) / K2 (DIP 12)
Interrupt	INTC In3

Description: A second 16550 UART is exposed on the DIP connector for communication with external devices (e.g., sensor modules, Bluetooth modules).

6.5 I2C Controller

Item	Value
Base Address	`0x4070_0000`
SCL Frequency	100 kHz (standard mode)
Input Glitch Filter	50 ns on SCL and SDA — per the I2C tSP spike-suppression spec
SCL / SDA Pins	DIP Pin 13 (L1) / Pin 14 (L2)
Pull-ups	Weak FPGA internal pull-ups enabled; external 4.7 kΩ to 3.3 V recommended for real devices
Interrupt	INTC In6

Description: An I2C master controls external I2C devices (sensors, EEPROMs, OLED displays). The bus uses open-drain signaling: any device may only pull the line low, and the pull-up resistor returns it high. Devices are addressed by their 7-bit I2C address. Interrupt routing is listed in Section 4.4. Use the Vitis `XIic` driver.

6.6 SPI Master

Item	Value
Base Address	`0x4080_0000`
SCLK	Software-programmable, ≈1.6 – 25 MHz (6.25 MHz at reset)
Clock Control	`0x4090_0000` — runtime clock setting; see `showcase/src/spi0.c` for the `spi0_set_clock()` helper
Slave Selects	2 — two devices can share the bus
Pins	DIP 35 SCLK (V3) · 36 MOSI (W5) · 37 MISO (V4) · 38 SS0 (U4) · 39 SS1 (V5)
Interrupt	INTC In7

Description: An SPI master controls external devices (displays, ADCs, flash modules). The interface uses full-duplex push-pull signaling and requires no pull-up resistors; the active device is selected by driving its SS line low. Use the Vitis `XSpi` driver. The serial clock is set at runtime through the clock-control block: select a lower rate for breadboard wiring or long cables, and a higher rate for short-wired display modules. A single call to `spi0_set_clock(hz)` takes effect immediately.

> **Note:** This is a second, independent SPI controller. Do not confuse it with `axi_quad_spi_0` (`0x4050_0000`), which is dedicated to the on-board QSPI boot flash. Firmware should select controllers by instance macro (`XPAR_SPI_0_BASEADDR` vs `XPAR_AXI_QUAD_SPI_0_BASEADDR`), never by generic `XPAR_XSPI_n_*` numbering.

6.7 Analog-to-Digital Converter (XADC)

Item	Value
Base Address	`0x4060_0000`
Resolution	12-bit
Conversion Rate	500 KSPS aggregate
Sequencer	Continuous scan over the 5 enabled channels

Enabled Channels:

Channel	Source	Description
VAUX4	DIP Pin 15 (G3/G2)	External analog input 0 (on-board divider: 0–3.3 V → 0–1 V)
VAUX12	DIP Pin 16 (H2/J2)	External analog input 1 (on-board divider: 0–3.3 V → 0–1 V)
Temperature	Internal	FPGA die temperature monitor
VCCINT	Internal	Core voltage monitor (1.0 V)
VCCAUX	Internal	Auxiliary voltage monitor (1.8 V)

Description: The Artix-7 built-in 12-bit ADC measures external analog signals and monitors internal FPGA temperature and supply voltages. External input pins pass through an on-board resistive voltage divider (2.32 kΩ / 1 kΩ, ratio ≈ 0.301), accepting up to 3.3 V at the DIP pin.

Effective Per-Channel Sampling Rate: The sequencer continuously cycles through all 5 enabled channels (VAUX4, VAUX12, Temperature, VCCINT, VCCAUX). The aggregate conversion rate is 500 KSPS, giving each channel an effective rate of 500 K ÷ 5 = 100 KSPS.

6.7.1 Analog Input Circuit

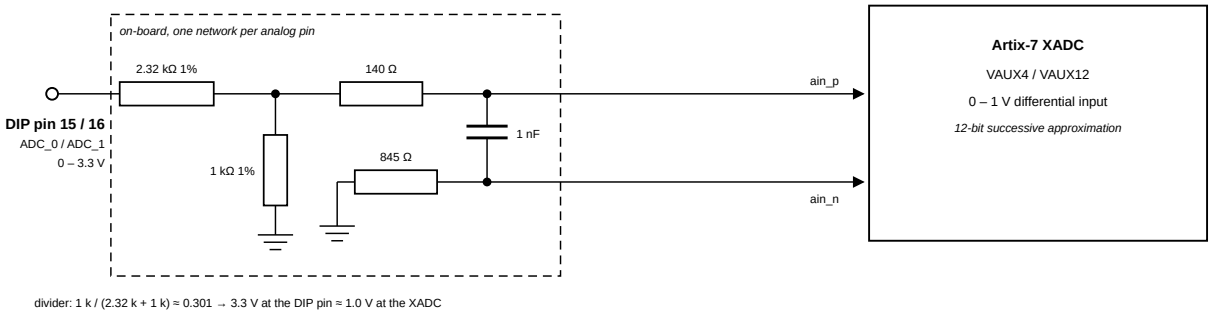


Figure 9: On-board voltage divider circuit for XADC analog inputs

The XADC expects an input range of 0–1 V. The board includes a resistive voltage divider (2.32 kΩ / 1 kΩ, all 1% precision) that scales the DIP pin voltage down to the FPGA's acceptable range. A 140 Ω series resistor and 1 nF capacitor are placed on the differential pair for filtering. The ADx_N path has an 845 Ω series resistor.

Parameter	Value
Max input voltage on DIP pin 15/16	**3.3 V** (relative to GND on pin 25)
Voltage at FPGA after divider	0–1 V
Divider ratio	1 kΩ / (2.32 kΩ + 1 kΩ) ≈ 0.301
Resistor tolerance	1%

> **Note:** Pins 15 and 16 are routed through on-board voltage dividers and are dedicated analog inputs; they are not available as digital I/O. Do not exceed 3.3 V on these pins.

7 JTAG Debug Mode

This chapter describes how to load and debug a MicroBlaze RISC-V application over JTAG using the AMD Vitis Unified IDE. JTAG loads the program directly into RAM and provides full debug control. Nothing is written to flash.

7.1 Prerequisites

- The hardware design as an `.xsa` file: use the prebuilt `release/top_wrapper.xsa`
- Vitis Unified IDE 2025.2 installed
- Cmod A7-35T board connected via its micro-USB cable (the cable carries power, UART, and JTAG)
- A serial terminal program (GTKTerm, PuTTY, or pyserial's miniterm) for the board's console output

7.2 Open the Workspace

Launch the Vitis Unified IDE and click **Set Workspace** on the Welcome page.

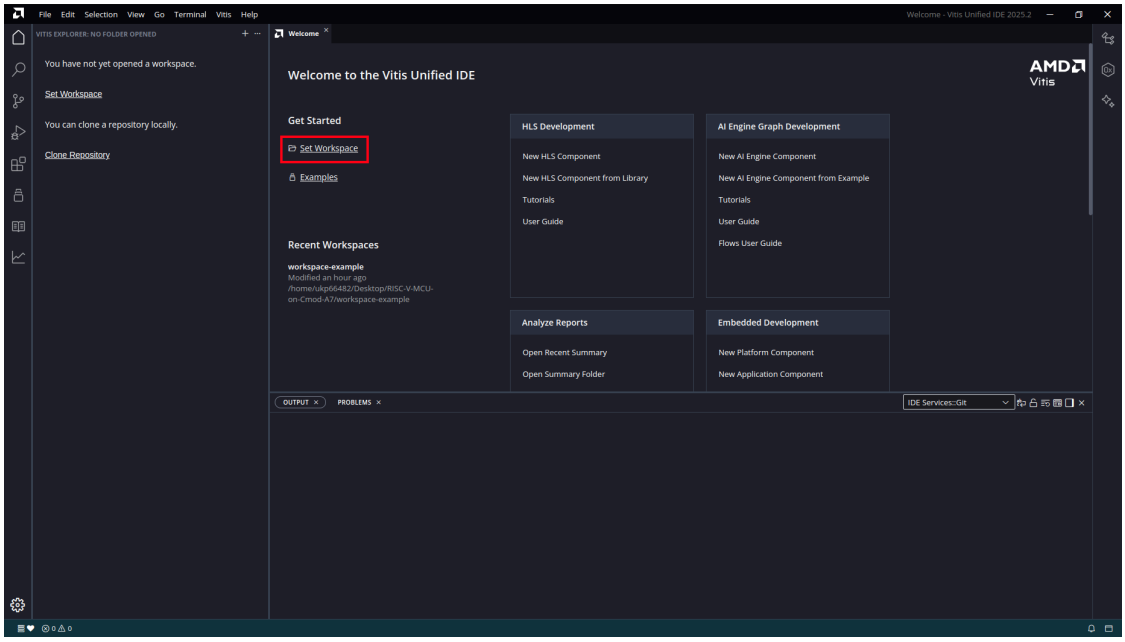


Figure 10: Set Workspace

Select the `workspace-example/` folder in the repository. The template paths in this chapter assume this location; any empty folder can also be used if the paths are adjusted accordingly.

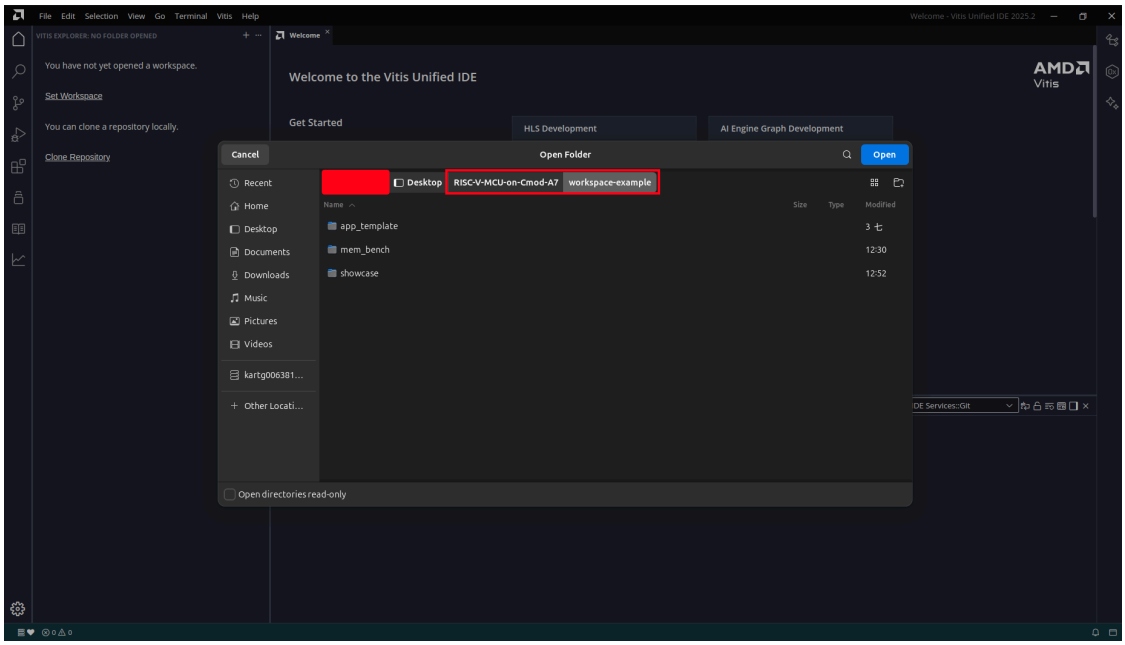


Figure 11: Open Folder

The first time a Vitis installation opens this workspace it may show an **Update Workspace** dialog. Click **Update**. This action refreshes only the workspace metadata and does not modify any component.

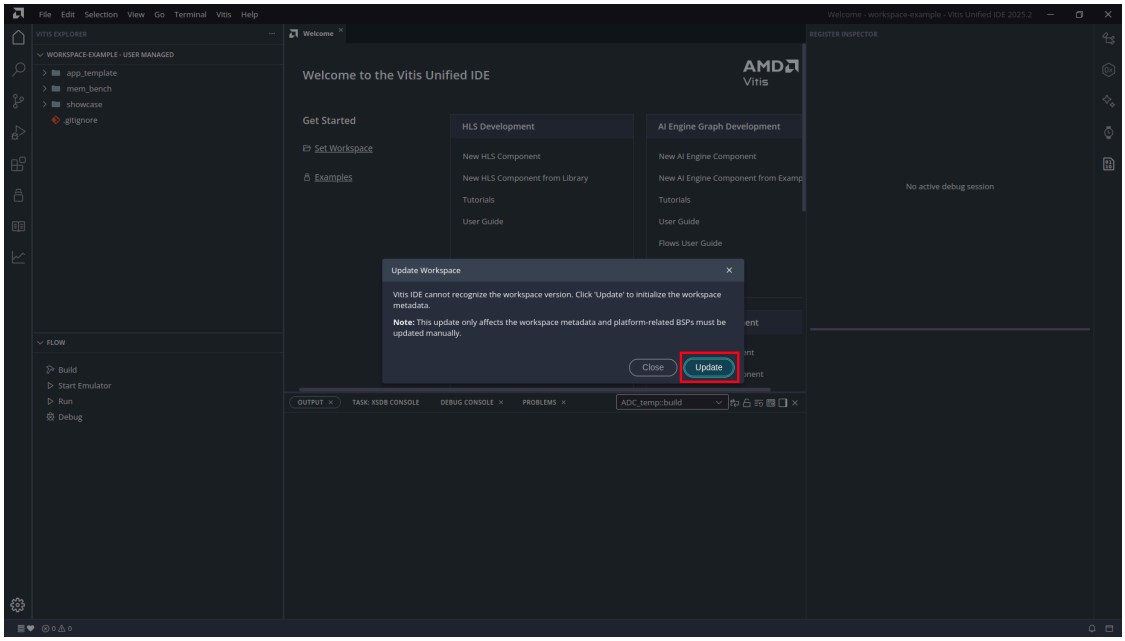


Figure 12: Update Workspace

7.3 Create a Platform

The platform wraps the hardware design and provides the driver layer (BSP). It is created once per workspace and reused by every application. If the workspace already contains a `platform` component, skip to Section 7.4.

7.3.1 New Platform Component

From the menu bar, select **File > New Component > Platform**.

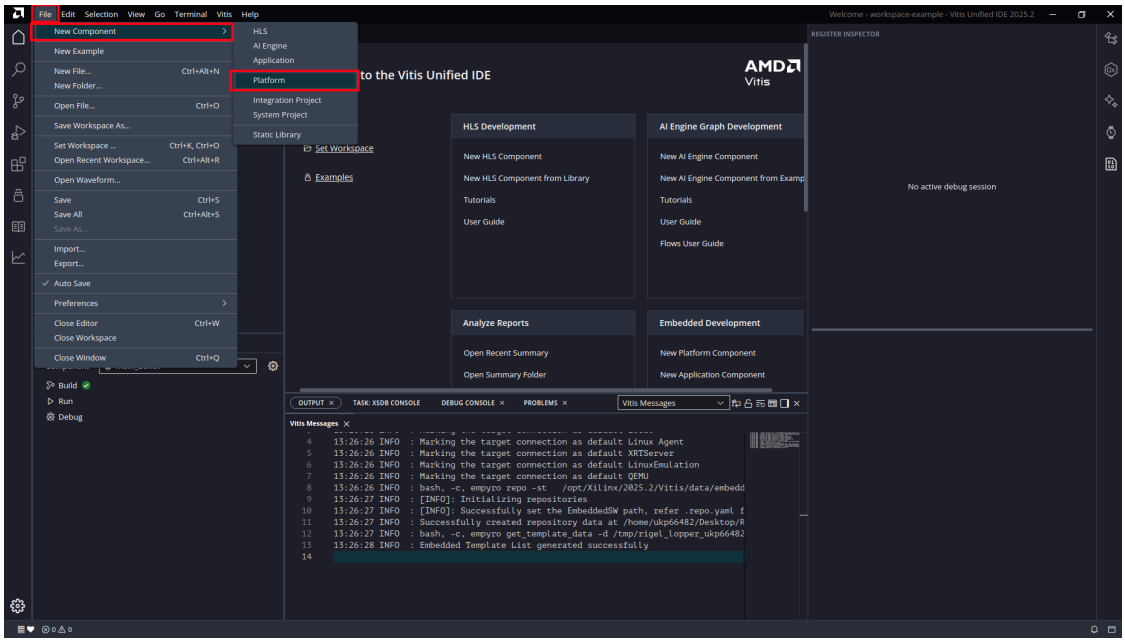


Figure 13: File > New Component > Platform

7.3.2 Platform Name and Location

Enter `platform` as the component name and keep the workspace directory as the location. Click **Next**.

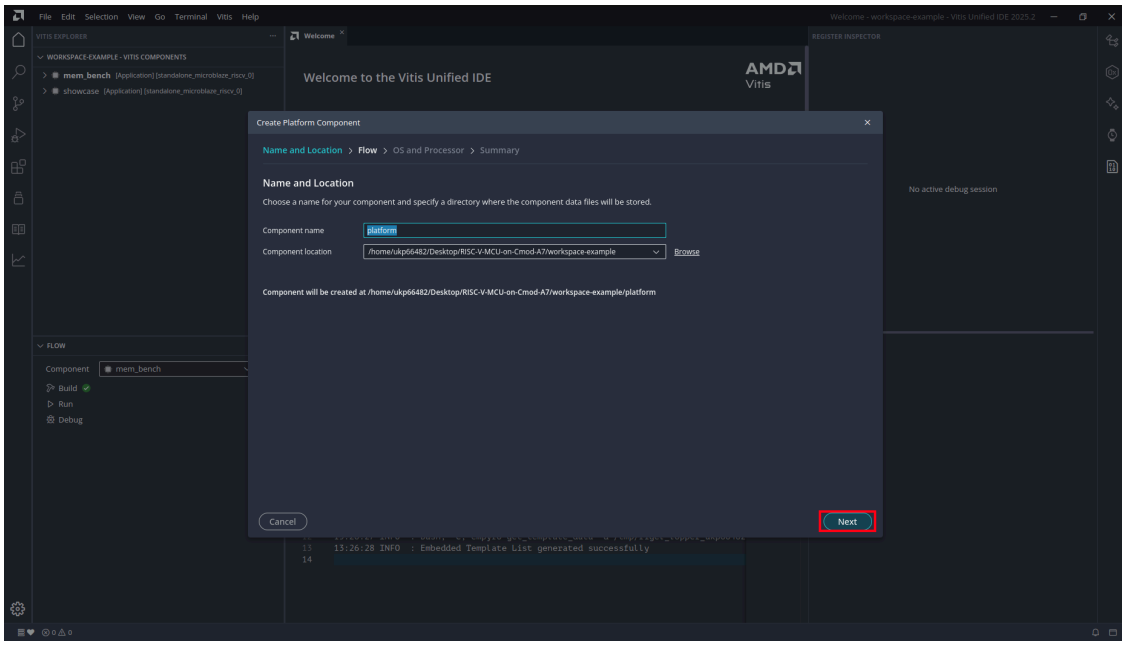


Figure 14: Platform Name and Location

7.3.3 Select the Hardware Design (XSA)

On the **Flow** page, select **Hardware Design** and click **Browse** next to the *Hardware Design (XSA) For Implementation* field.

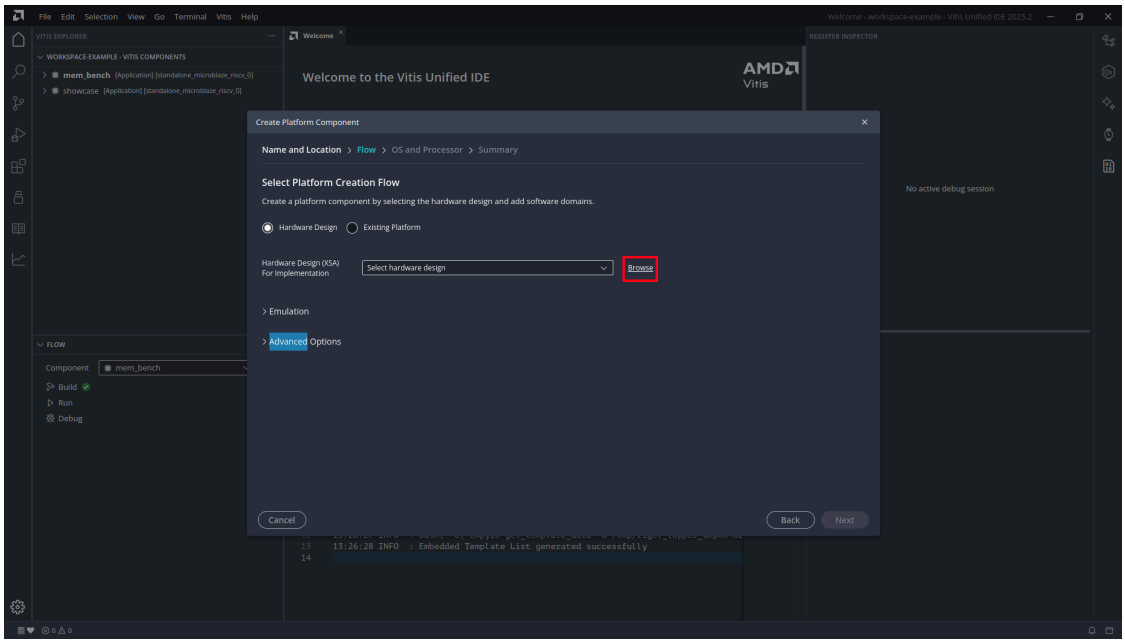


Figure 15: Select Platform Creation Flow

Select the prebuilt `release/top_wrapper.xsa` from the repository.

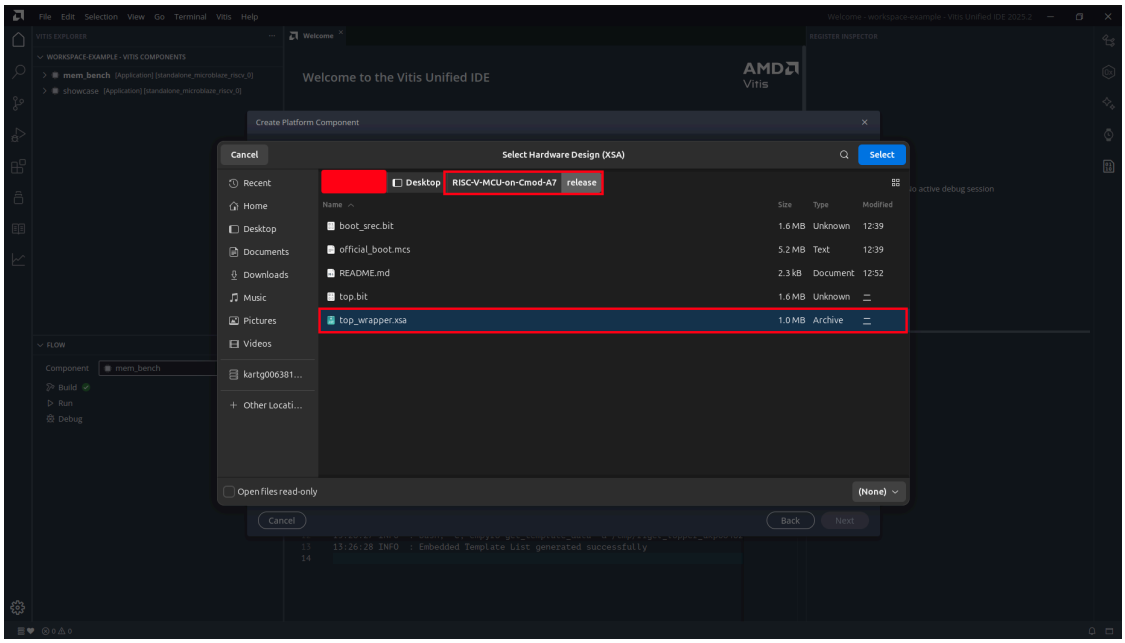


Figure 16: Select top_wrapper.xsa

7.3.4 Select Operating System and Processor

- **Operating system**: `standalone`
- **Processor**: `microblaze_riscv_0`

Click **Next**.

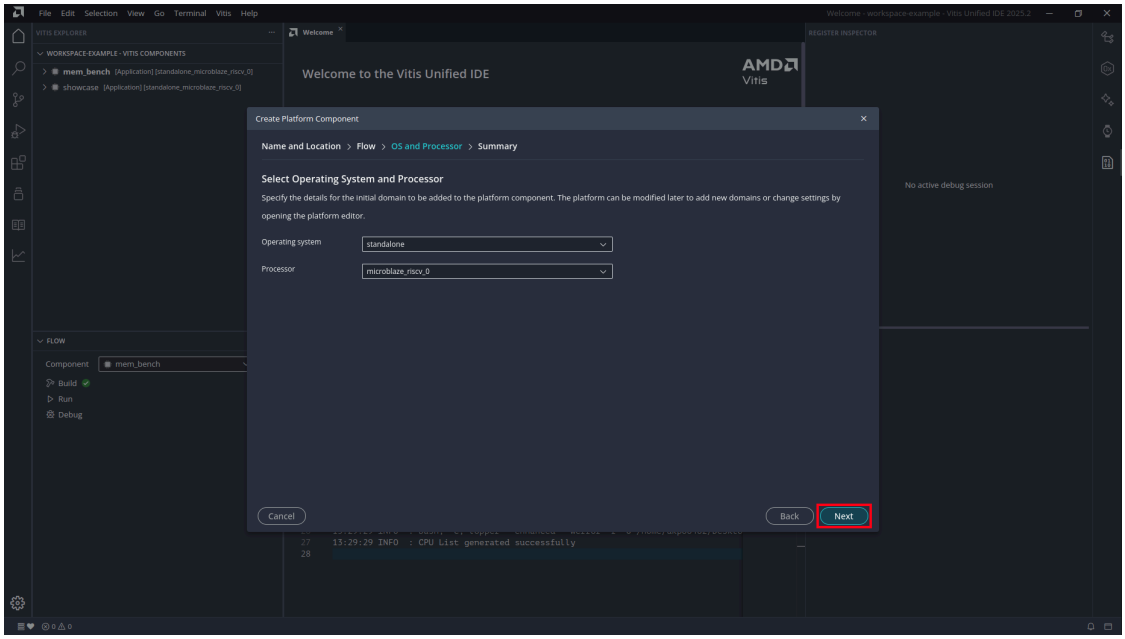


Figure 17: OS and Processor

7.3.5 Platform Summary

Review the settings and click **Finish**.

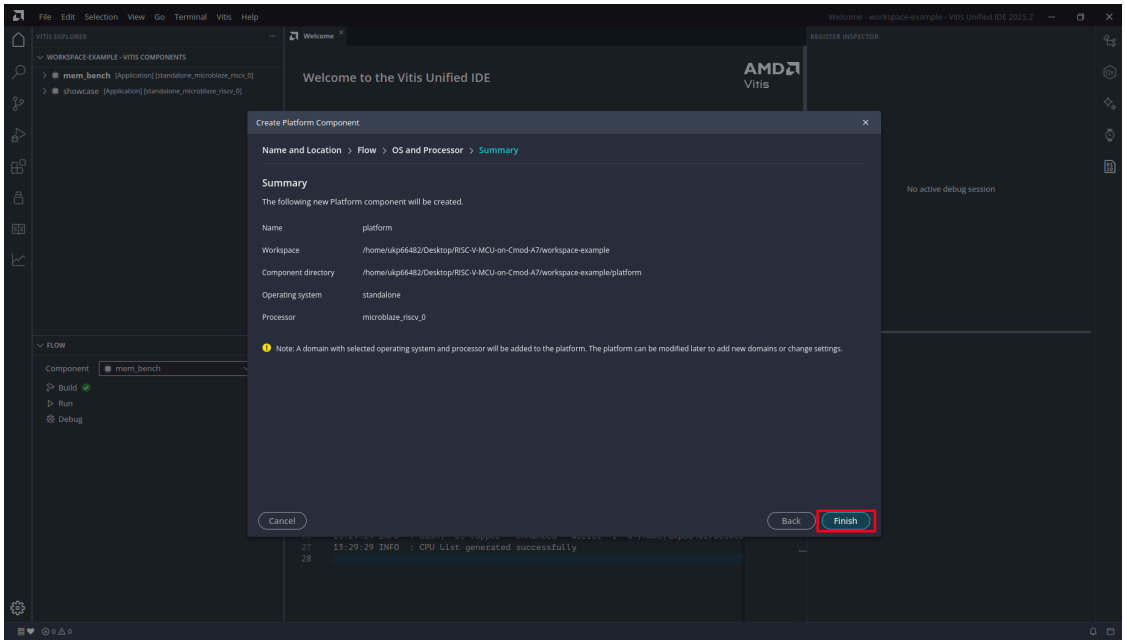


Figure 18: Platform Summary

Vitis generates the platform component. When it finishes, the Explorer shows the `platform` component with its `Sources` and `Output` trees, and the notification *“Created platform: platform”* appears.

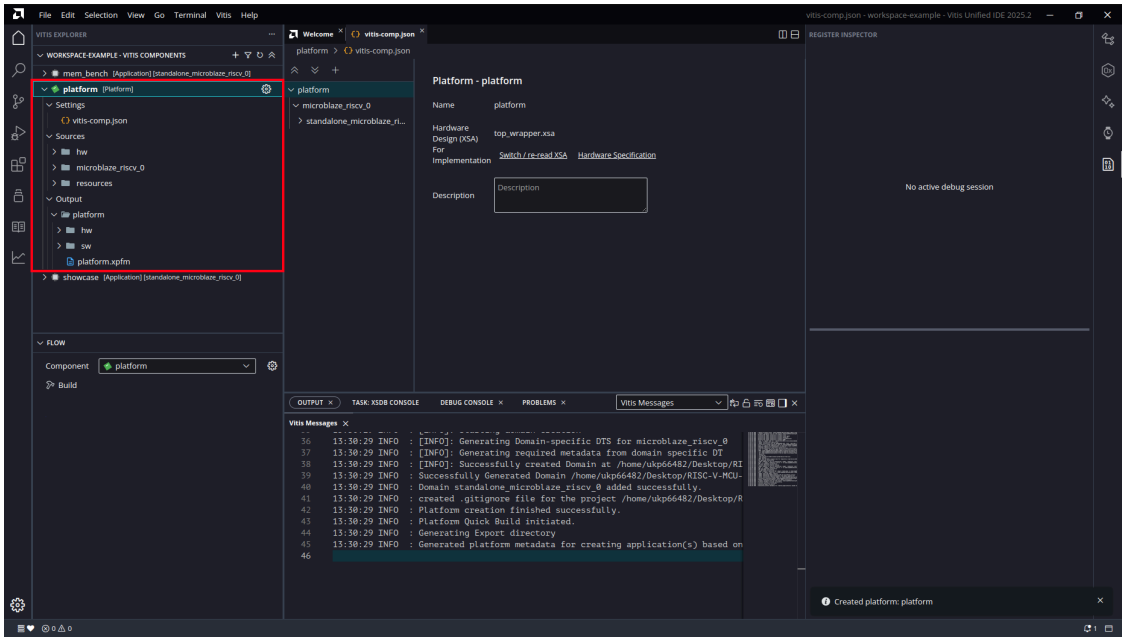


Figure 19: Platform Created

7.4 Create an Application

7.4.1 Create from Template

Open the **Examples** view from the left activity bar. Under *Embedded Software Examples*, select **Hello World** and click the **+** (Create Application Component from Template) button.

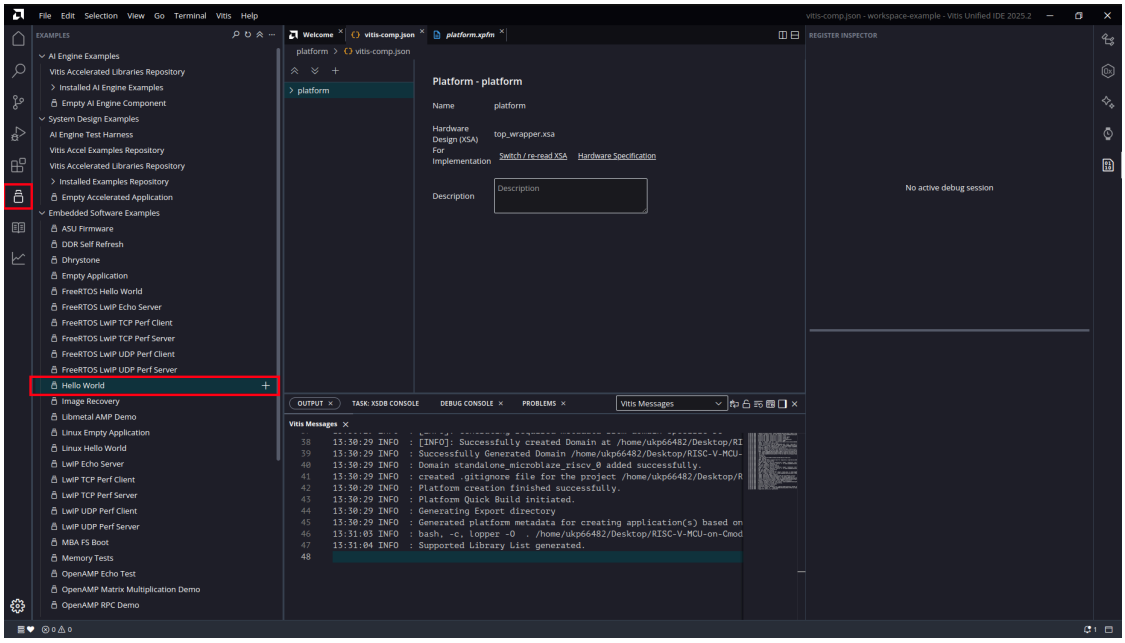


Figure 20: Hello World Template

7.4.2 Application Name and Location

Enter your application name (this chapter uses `test`). Click **Next**.

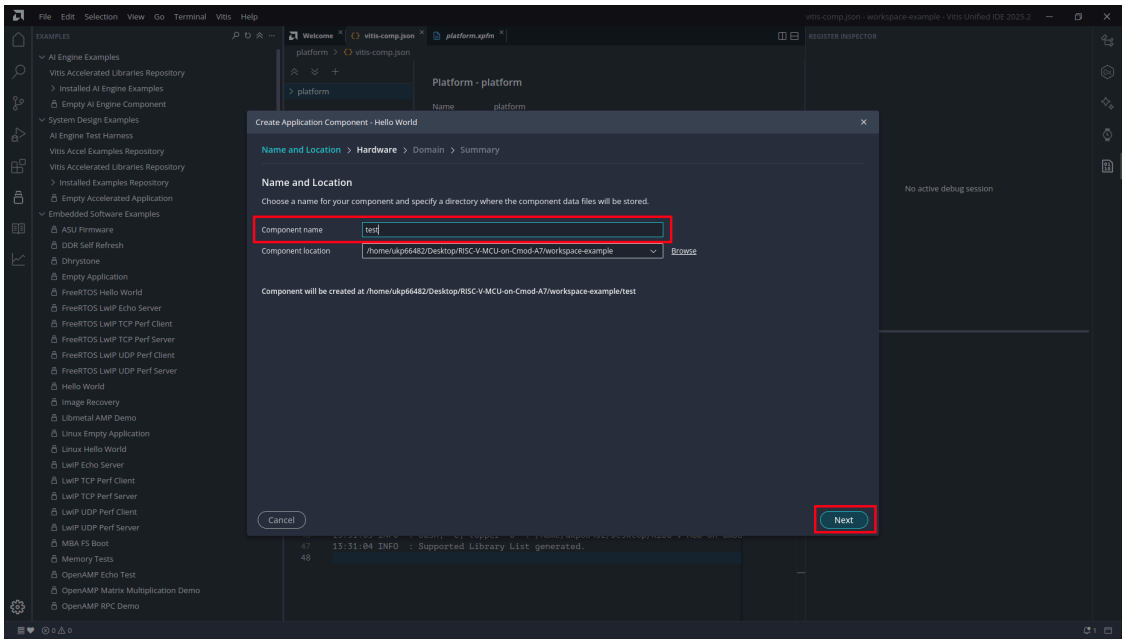


Figure 21: Application Name

7.4.3 Select the Platform

Select the `platform` component created in Section 7.3 (Board: `cmod_a7-35t`). Click **Next**.

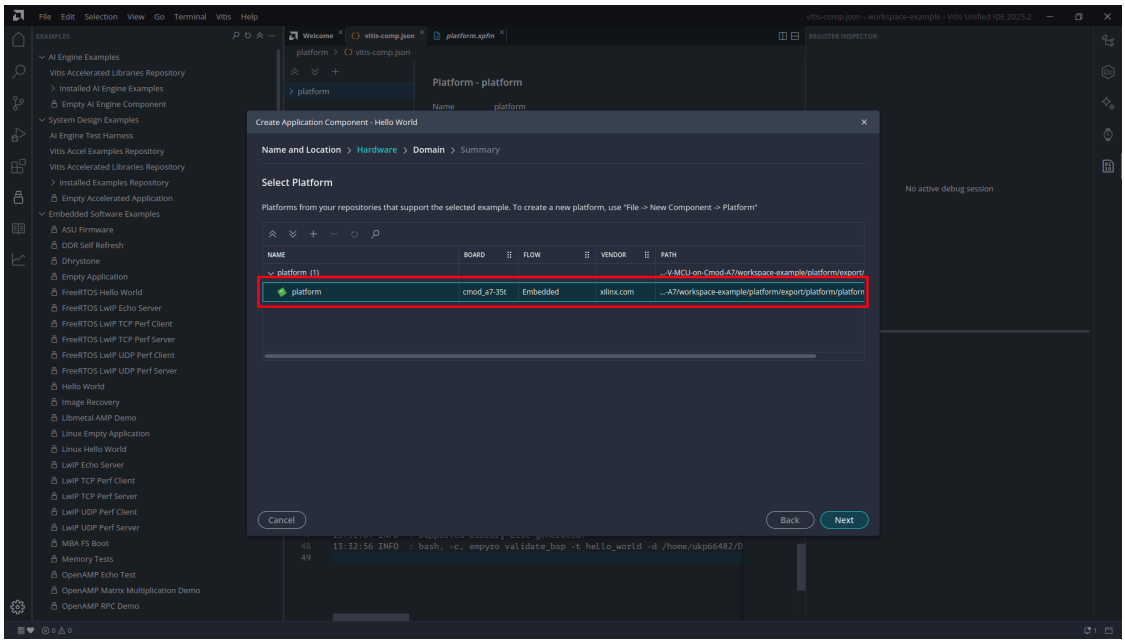


Figure 22: Select Platform

7.4.4 Select the Domain

Select `standalone_microblaze_riscv_0`. Click **Next**.

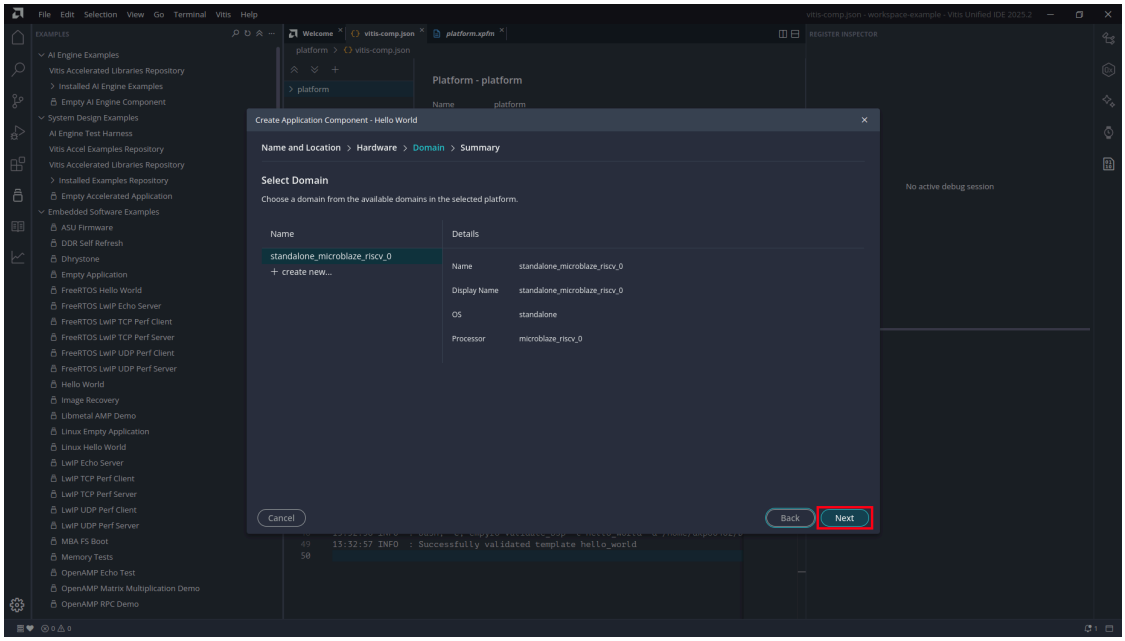


Figure 23: Select Domain

7.4.5 Application Summary

Review the settings and click **Finish**.

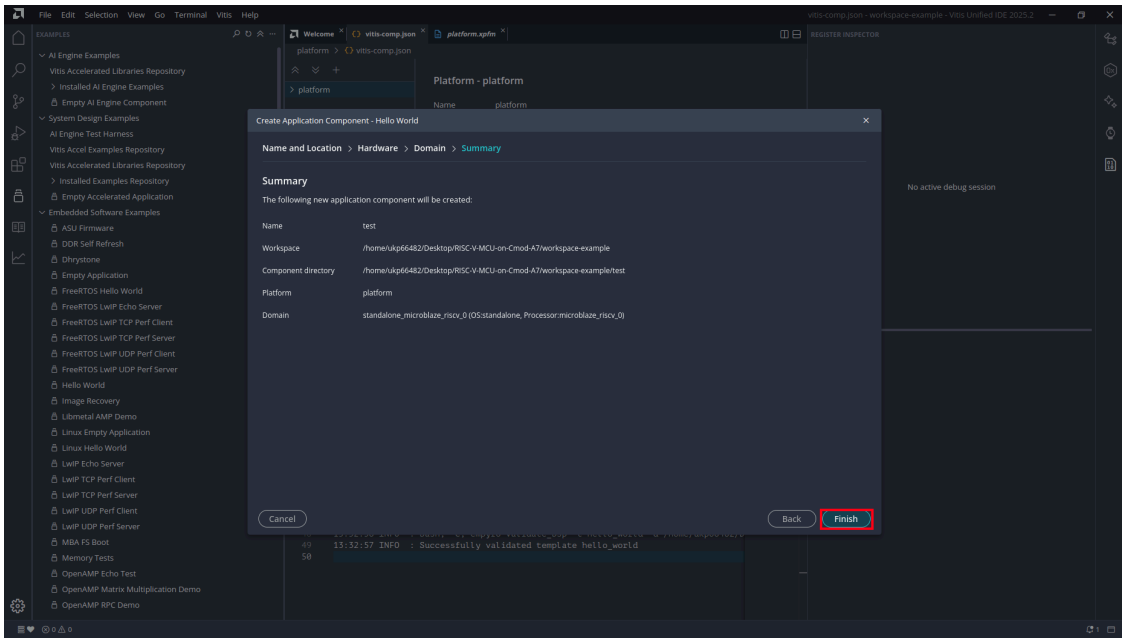


Figure 24: Application Summary

The new component appears in the Explorer. Its `src/` folder holds the template's generated files: `helloworld.c`, `lscript.ld`, `platform.c`, and `platform.h`.

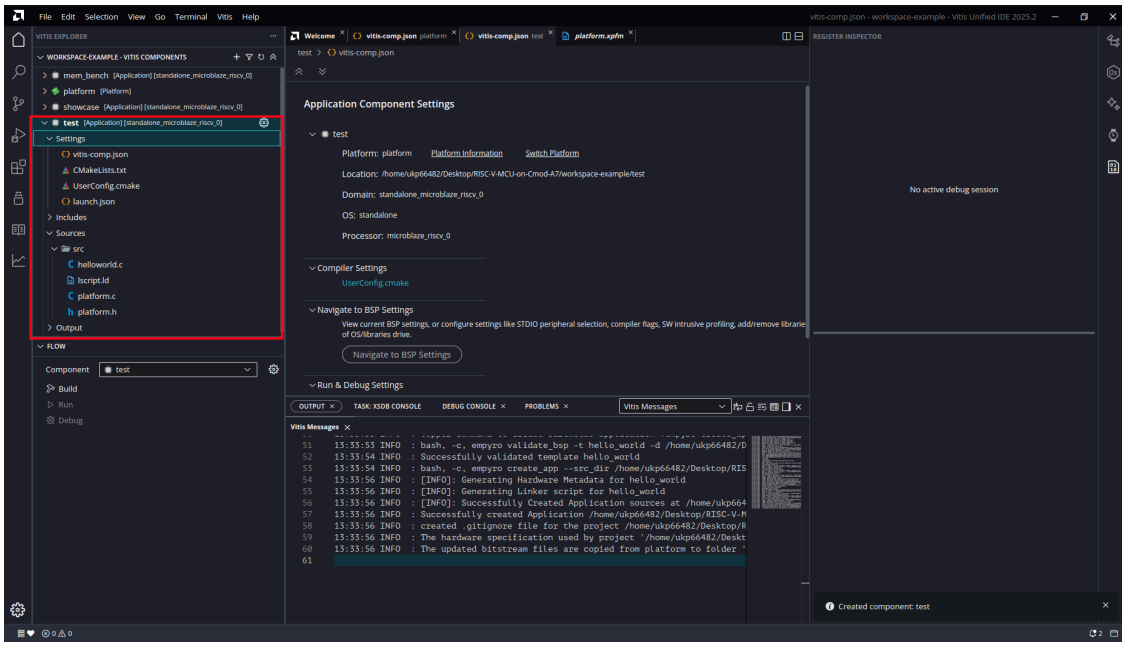


Figure 25: Application Created

7.5 Replace the Sources with the Course Template

The generated Hello World sources are replaced entirely by the course template. The template `main.c` starts with `mcu_init()`, and its `lscript.ld` places code in the 512 KB SRAM, places the stack in DTCM, and provides the `ITCM_FUNC`/`DTCM_DATA` placement attributes.

7.5.1 Delete the Generated Files

In the Explorer, select all four files under `src/` (`helloworld.c`, `lscript.ld`, `platform.c`, `platform.h`), right-click, and choose **Delete**.

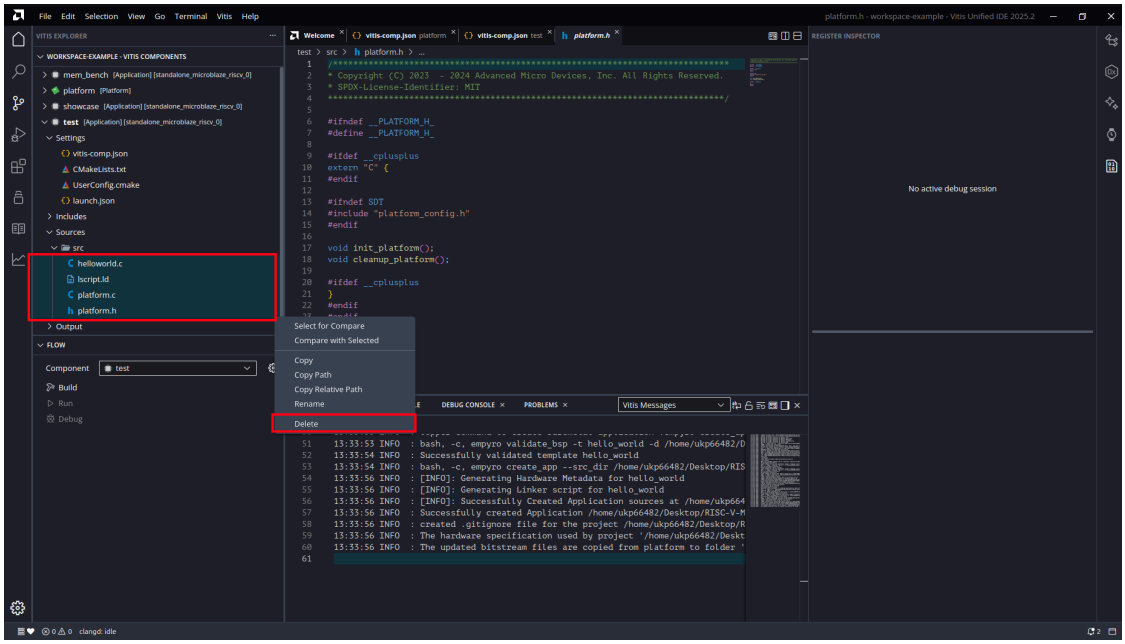


Figure 26: Delete Generated Sources

7.5.2 Import the Template Files

Right-click the `src/` folder and choose **Import > Files…**

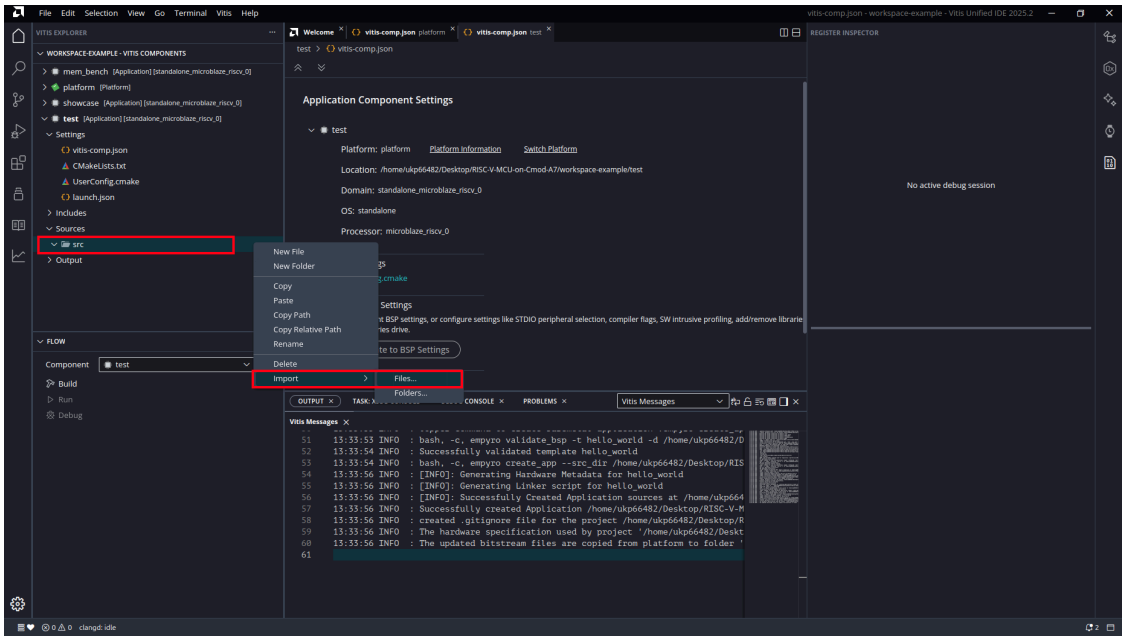


Figure 27: Import Files

Navigate to `workspace-example/app_template/src/` and select both `main.c` and `lscript.ld`. Click **Open**.

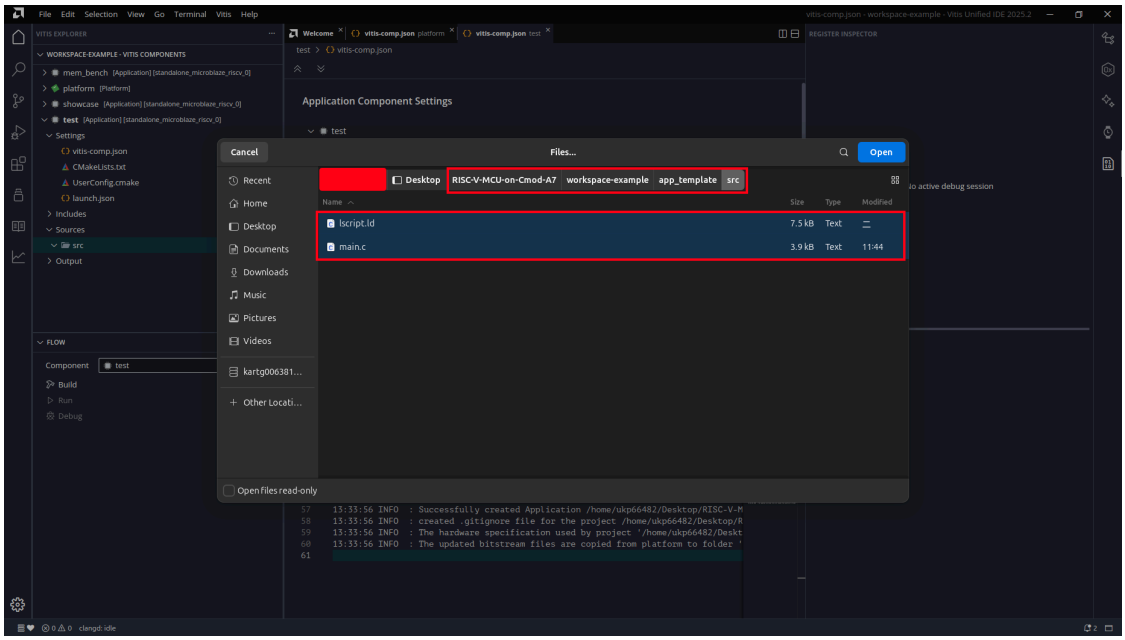


Figure 28: Select Template Files

Afterwards `src/` contains exactly `main.c` and `lscript.ld`. Files inside `src/` are compiled automatically; no build configuration is needed.

> **Note:** The `mcu_init();` call must remain the first line of `main()`. It copies `ITCM_FUNC` code into the ITCM, zeroes the DTCM, and sets the USB UART to 115200.

The demo loop cycles the on-board RGB LED and prints a status line about once a second.

7.5.3 About the Linker Script

The imported `lscript.ld` defines three memory regions: `sram` (512 KB), `itcm` (32 KB), and `dtcm` (64 KB). It maps everything to SRAM except the stack (DTCM) and `ITCM_FUNC` code (ITCM). The component settings page *Linker Script* can display the regions, but treat it as read-only: it understands only part of the hand-written script (an incomplete section list is expected), and letting it regenerate the script would discard the ITCM/DTCM layout. To resize the stack or heap, edit `_STACK_SIZE` / `_HEAP_SIZE` at the top of `lscript.ld` instead.

7.6 Set the Console UART (BSP)

The BSP determines which UART carries `xil_printf` output. The default is `uart_1` (the DIP-header pins); with the default setting, no output appears on the USB console.

Open the platform's **Settings > vitis-comp.json**, navigate to **standalone** (Board Support Package > microblaze_riscv_0 > standalone). The configuration table shows `standalone_stdin` and `standalone_stdout` set to `uart_1`:

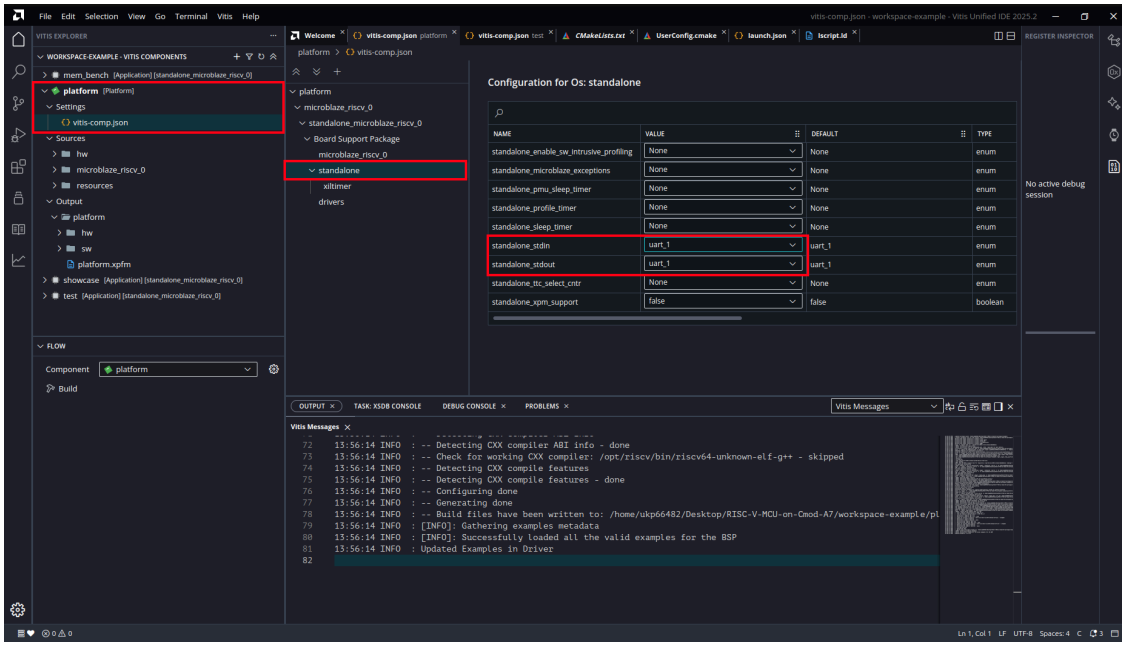


Figure 29: BSP Defaults to uart_1

Change both to `uart_USB`. The output log reports the domain being reconfigured.

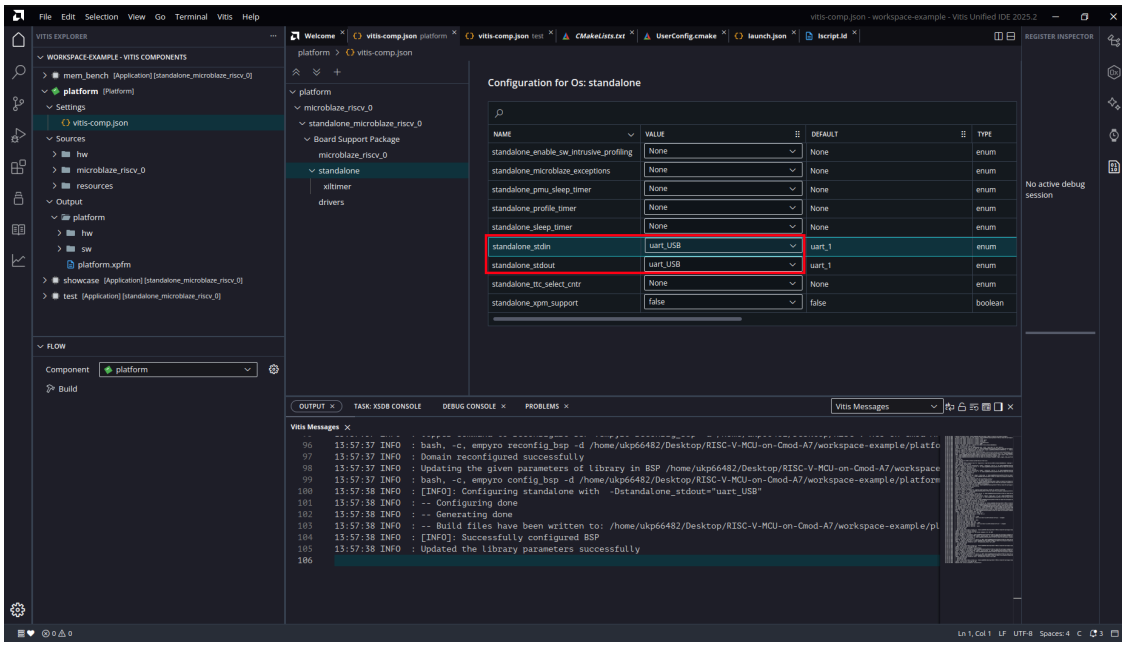


Figure 30: stdin/stdout Set to uart_USB

> **Note:** If the platform BSP is ever regenerated (for example after changing another BSP option), re-check this page; regeneration can reset stdin/stdout to `uart_1`. After changing it, rebuild both the platform and the application; the setting is compiled into the application's BSP library.

7.7 Build

7.7.1 Build the Platform

In the **FLOW** panel, select the `platform` component and click **Build**. Wait for *"Platform Build Finished successfully"*.

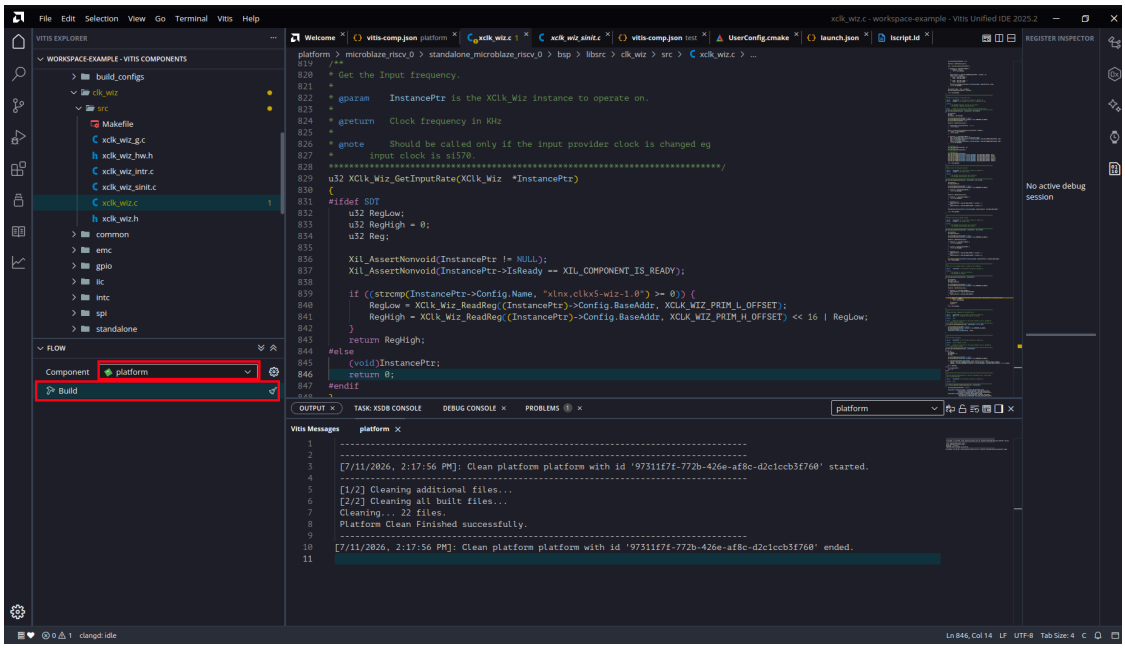


Figure 31: Build Platform

7.7.2 Build the Application

Switch the FLOW component to your application and click **Build**.

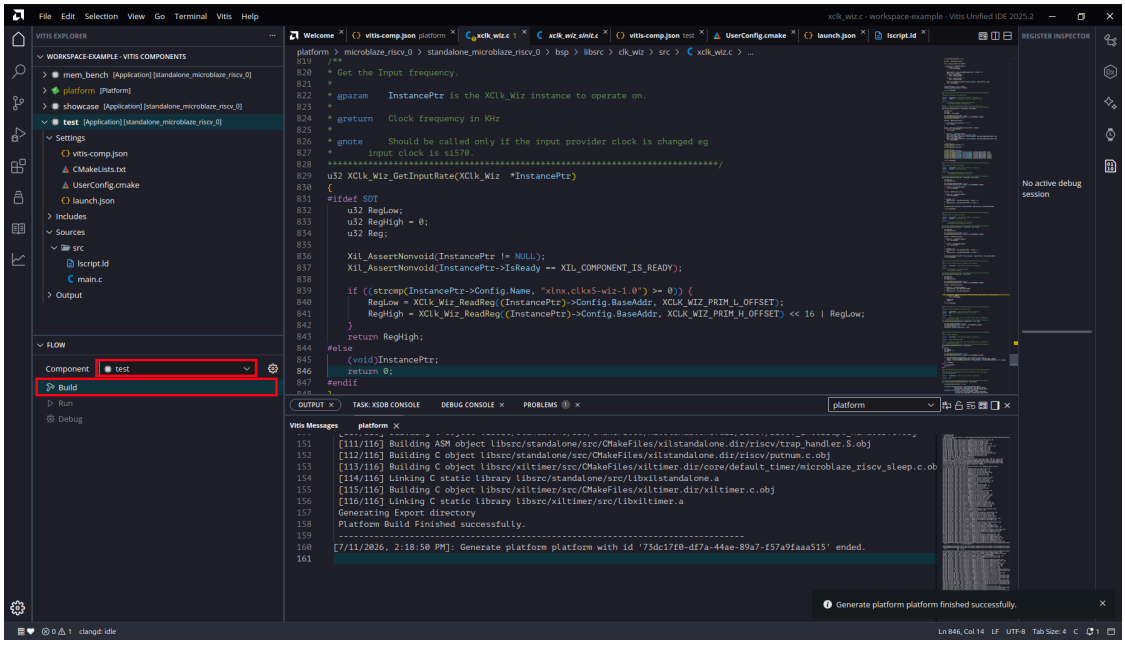


Figure 32: Build Application

The first application build asks how platform builds should be handled. Select **Always build platform with application** and click **Save in Workspace Preference**. BSP changes (such as Section 7.6) then propagate automatically on every build.

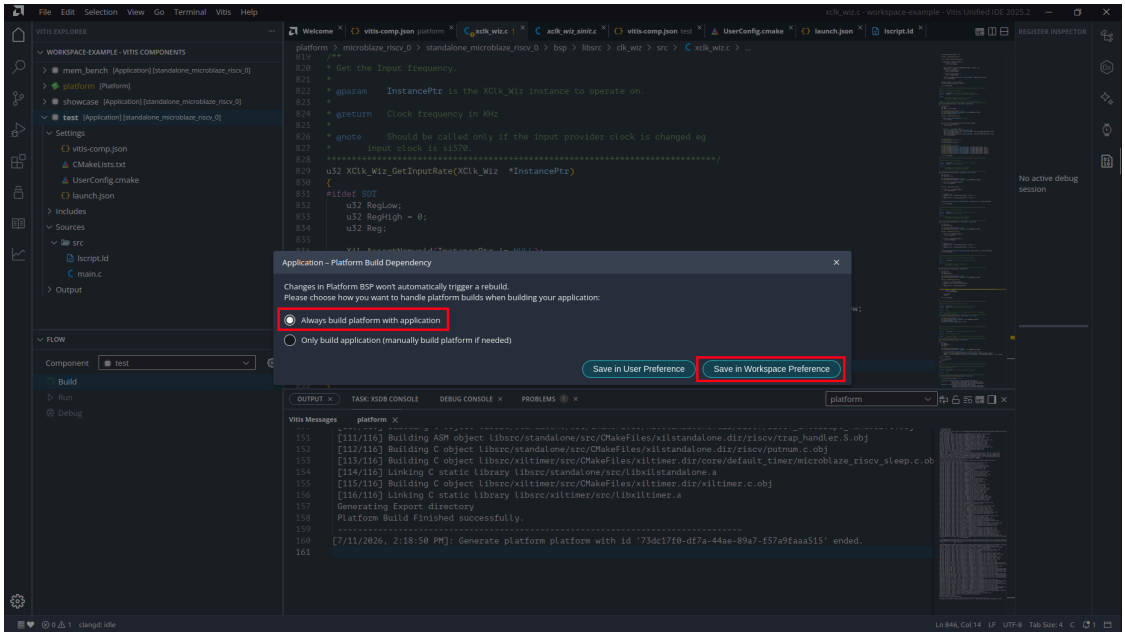


Figure 33: Platform Build Dependency

The build produces `<workspace>/test/build/test.elf`.

> **Note:** During the build, Vitis automatically links in the BSP's `boot.S` and the toolchain C library's startup object `crt0.o`: `boot.S` runs first after reset (initializes registers, stack pointer, and trap vector), and `crt0.o` clears `.bss` before calling `main()`.

7.8 Debug over JTAG

7.8.1 Start a Debug Session

In the **FLOW** panel, click **Debug**. Vitis programs the FPGA over JTAG, loads the ELF into memory, and pauses execution at `main()`.

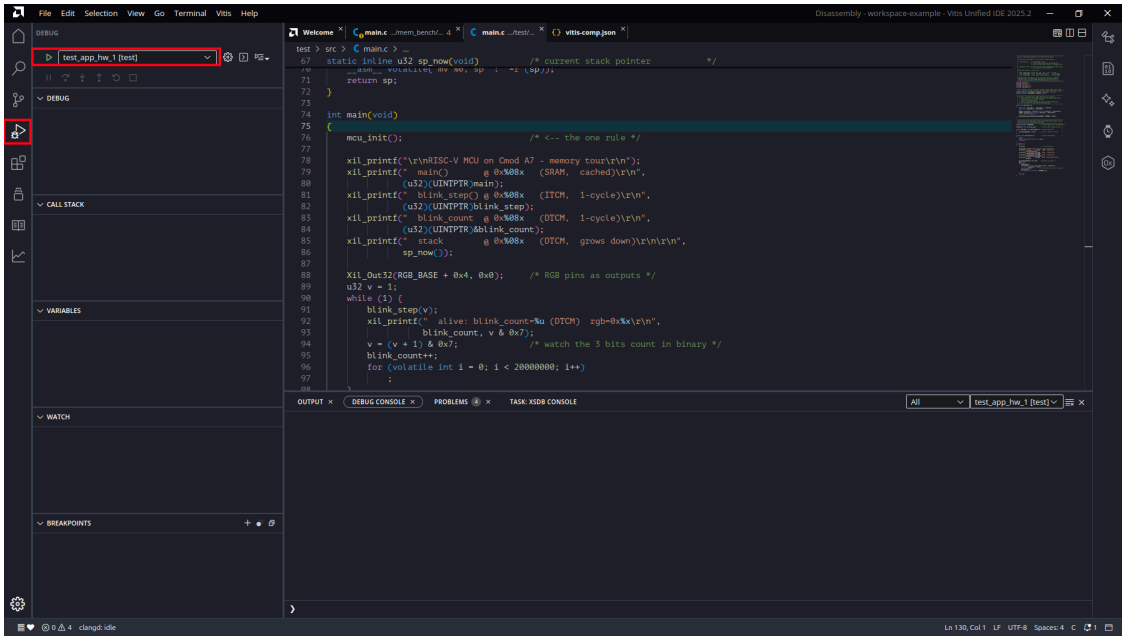


Figure 34: Debug Launched

The Debug view shows the target (*RISC-V at USER2*, *Hart #0 — PAUSED ON BREAKPOINT*), the call stack, local variables, and breakpoints. Note the call stack address: `main()` is located at `0x600009EC`, in SRAM, exactly where the linker script placed it.

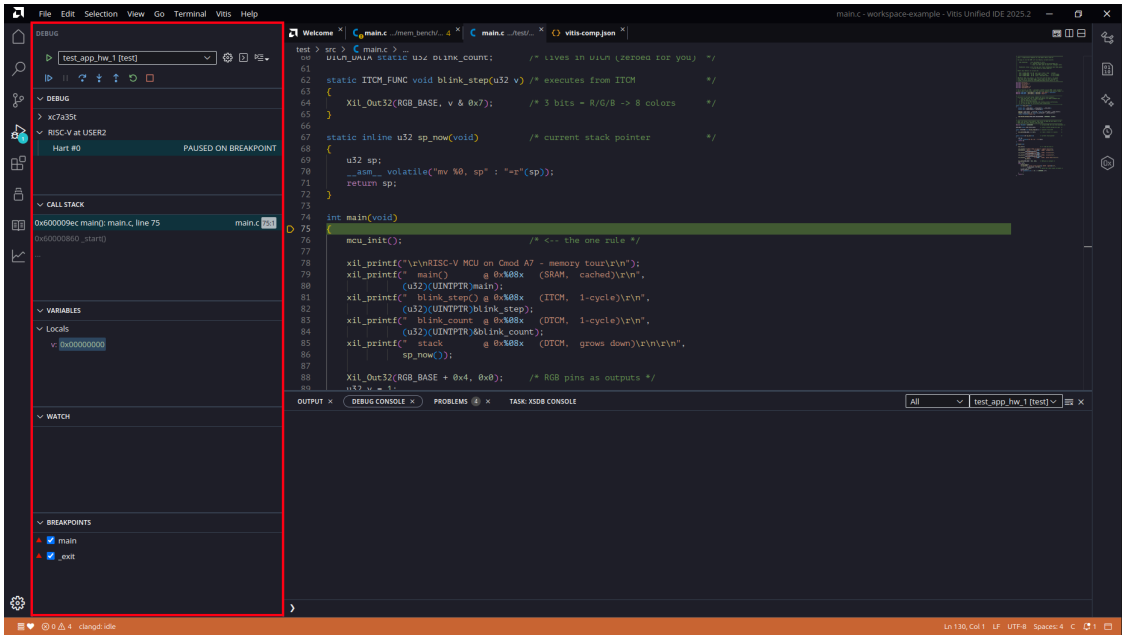


Figure 35: Paused at main()

7.8.2 Open a Serial Terminal, Then Continue

Open a serial terminal before resuming execution. The board enumerates as two serial ports; the UART is usually the higher-numbered port. Set the terminal to 115200 8N1.

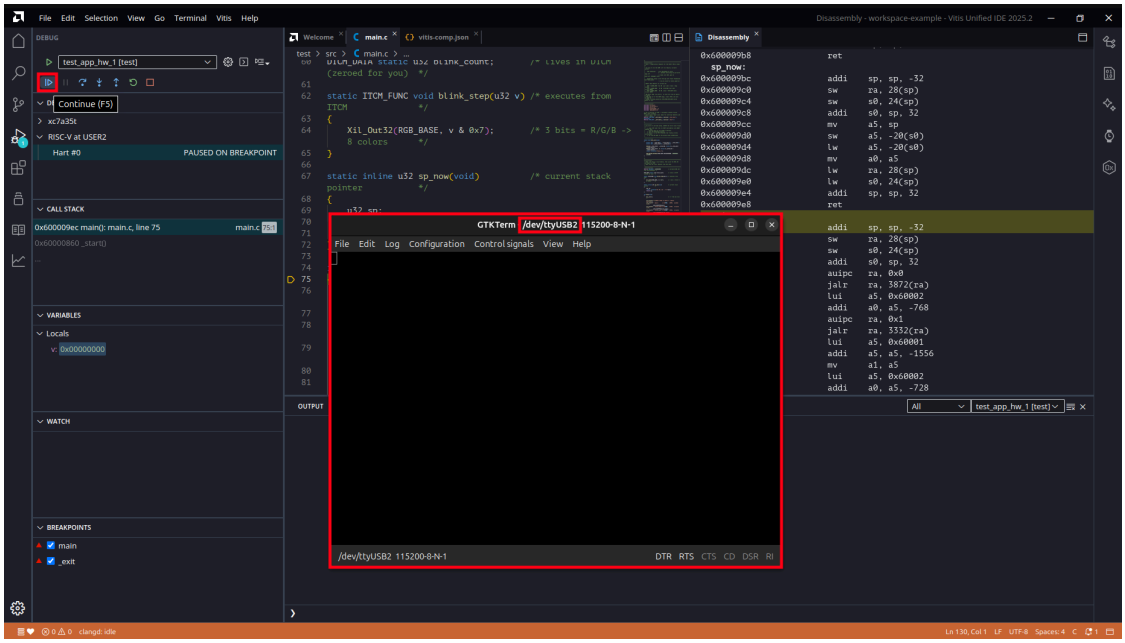


Figure 36: GTKTerm on the Board UART

Click **Continue (F5)**. The application starts: the memory tour prints once, followed by a status line about once a second, and the on-board RGB LED cycles through its colors.

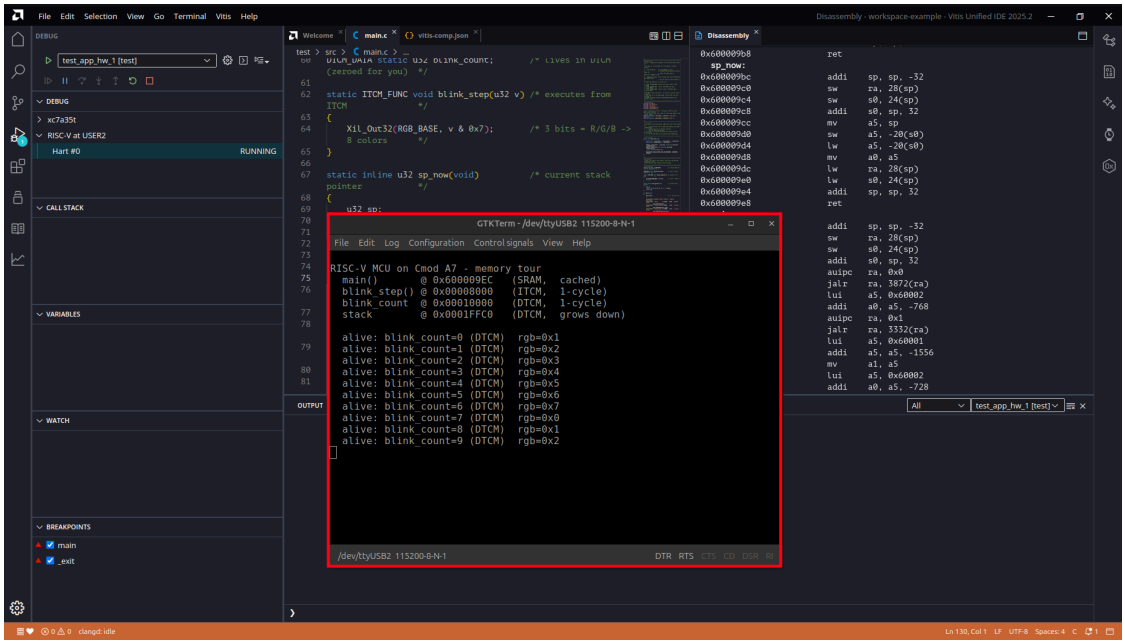


Figure 37: Memory Tour and Status Output

The memory tour output confirms that the memory layout is correct:

```
RISC-V MCU on Cmod A7 - memory tour
  main()       @ 0x600009EC   (SRAM,  cached)
  blink_step() @ 0x00008000   (ITCM,  1-cycle)
  blink_count  @ 0x00010000   (DTCM,  1-cycle)
  stack        @ 0x0001FFC0   (DTCM,  grows down)
```

Each line matches the memory map in the datasheet: code in SRAM, fast code in ITCM, fast data and the stack in DTCM. If no output appears on the terminal, see Section 7.10.

7.8.3 Breakpoints and Stepping

Click in the gutter next to a line number to set a breakpoint. The toolbar provides **Continue**, **Step Over**, **Step Into**, and **Step Out**. The left panel shows the call stack, variables, and watch expressions while the target is paused.

To run the application without the debugger, use **Run** in the FLOW panel instead. The loading sequence is the same, but execution does not pause at `main()`.

7.9 Inspection Tools

While a debug session is active, the **View** menu provides three inspectors:

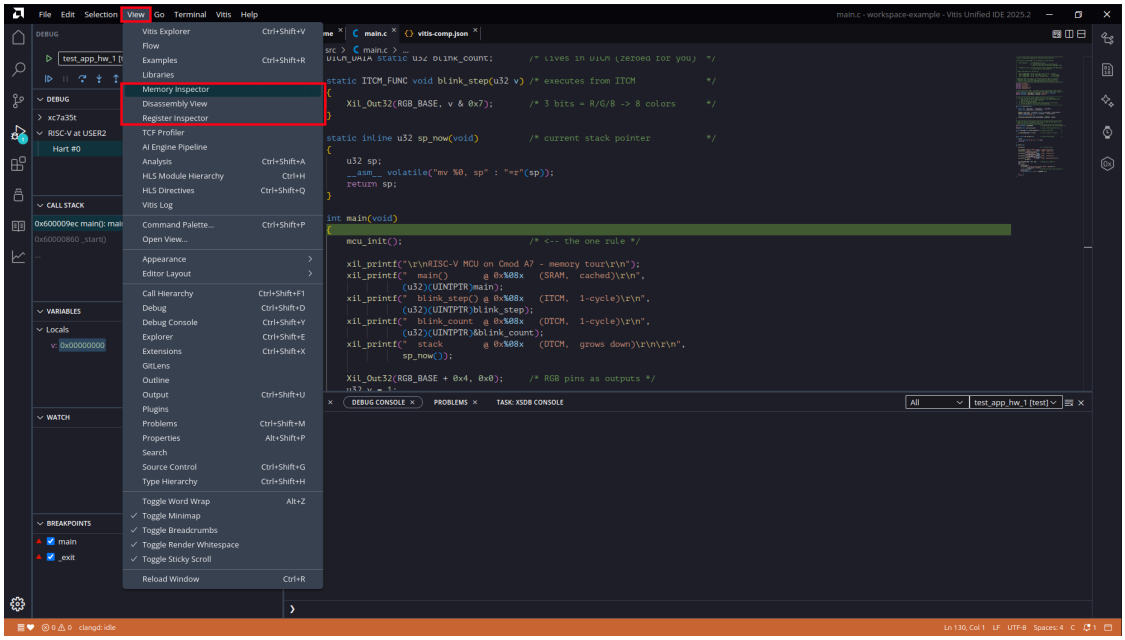


Figure 38: View Menu

7.9.1 Memory Inspector

Enter an address to inspect memory directly, for example `0x60000000` (the application image in SRAM), `0x8000` (ITCM), or `0x10000` (DTCM).

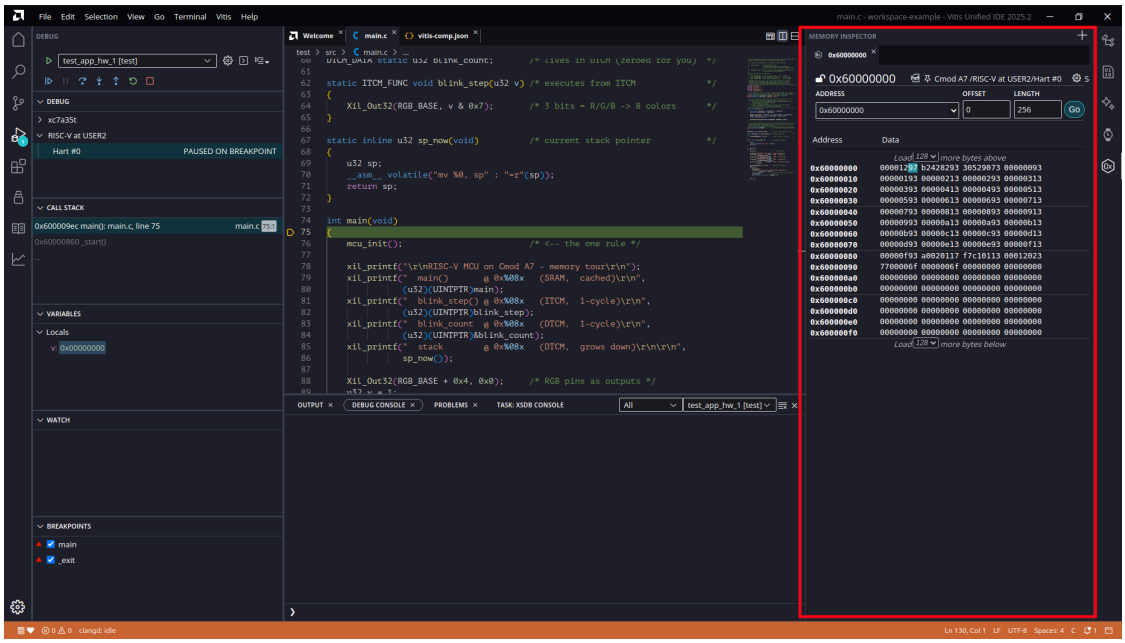


Figure 39: Memory Inspector

7.9.2 Register Inspector

The Register Inspector shows all processor registers with live values. With the target paused at `main()`, `sp` reads `0x00020000`, the top of the DTCM, matching the linker script's stack placement.

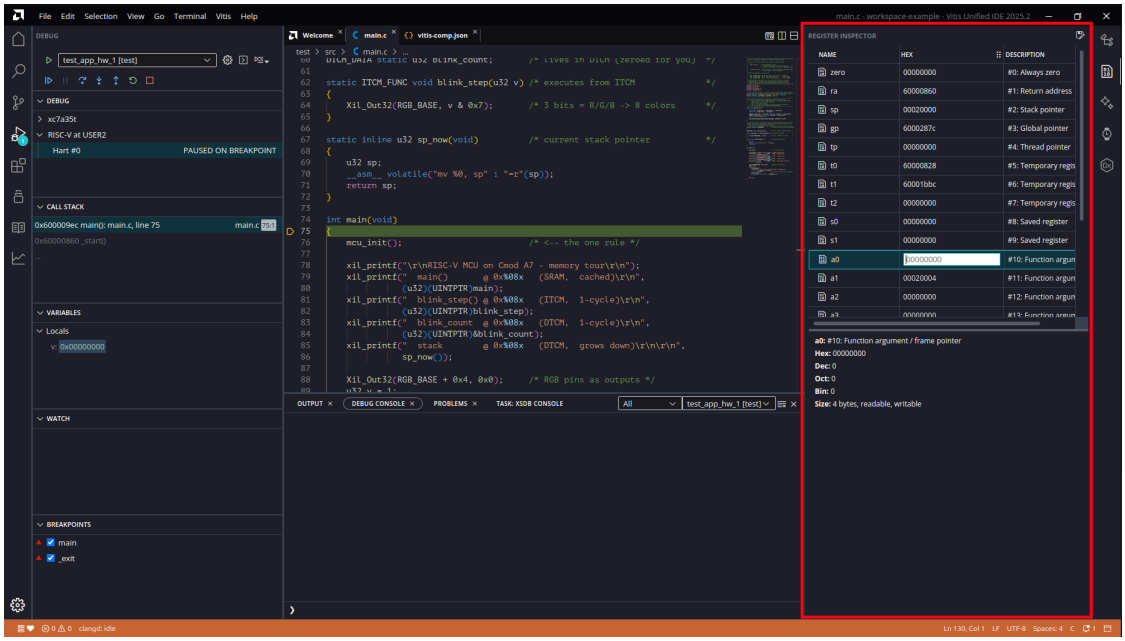


Figure 40: Register Inspector

7.9.3 Disassembly View

The Disassembly view shows the executing machine code with source interleaving. The instruction addresses around `main()` all fall in `0x6000_xxxx`; the program runs from SRAM.

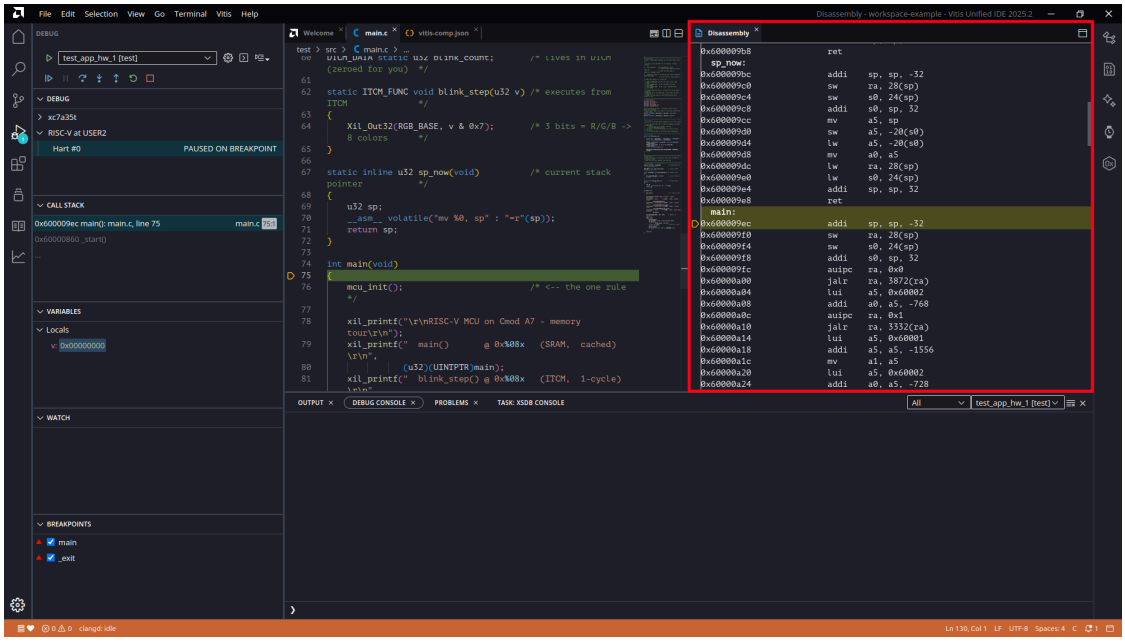


Figure 41: Disassembly

7.10 Troubleshooting

Platform build fails with `error: passing argument 2 of 'strcmp' ... [-Wint-conversion]` in `xclk_wiz.c` (reported as *"Error in generating platform"*). Two conditions combine to cause this error:

- The clock-management driver source that ships with Vitis 2025.2 (`clk_wiz` v1_10) contains a misplaced parenthesis in `xclk_wiz.c` line 839.
- If another RISC-V toolchain with GCC 14 or newer is on your PATH (for example one installed at `/opt/riscv` for assembly coursework), the BSP build may select it instead of the Vitis-bundled compiler. GCC 14+ treats this construct as an error; the bundled compiler treats it as a warning.

The offending line as displayed in the IDE:

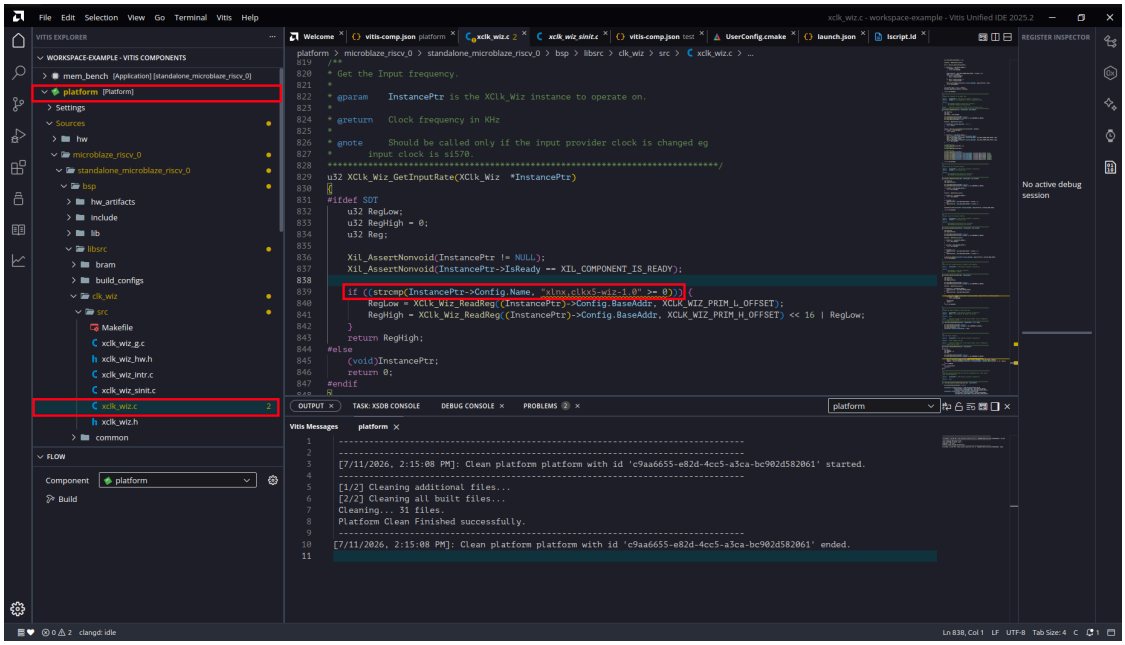


Figure 42: clk_wiz Driver Bug

Fix the source (this fix works with either compiler): change line 839 from

```
strcmp(InstancePtr->Config.Name, "xlnx,clkx5-wiz-1.0" >= 0)
```

to

```
strcmp(InstancePtr->Config.Name, "xlnx,clkx5-wiz-1.0") >= 0
```

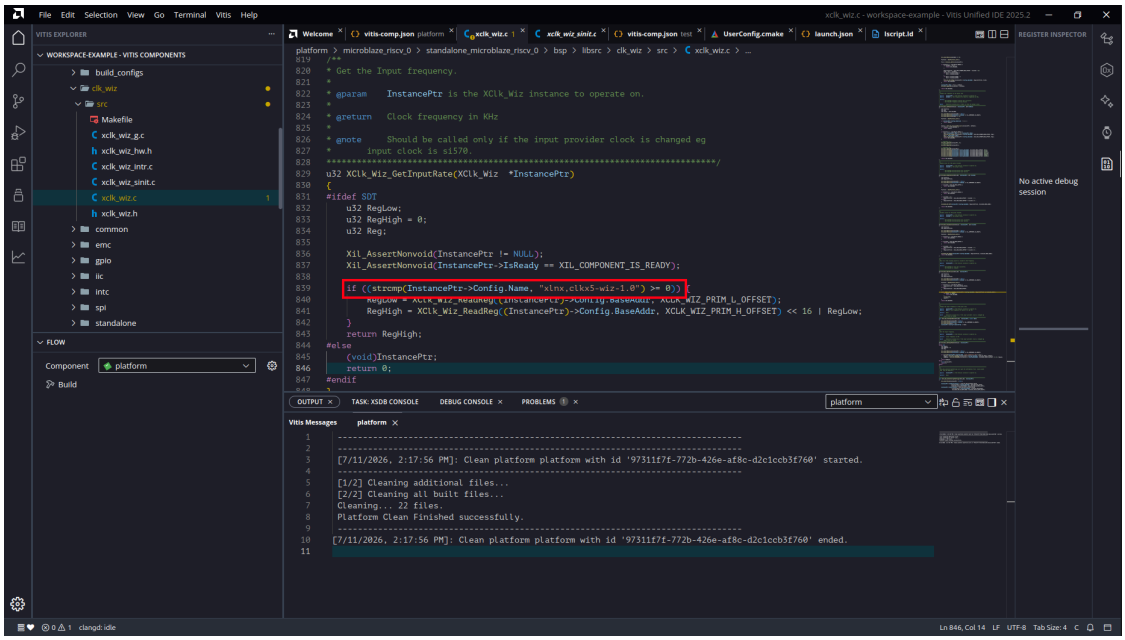


Figure 43: After the Fix

Then rebuild the platform. Apply the same one-line fix to the Vitis installation copy at `<install>/2025.2/data/embeddedsw/XilinxProcessorIPLib/drivers/clk_wiz_v1_10/src/xclk_wiz.c`, where `<install>` is the Xilinx installation directory (`/opt/Xilinx` by default on Linux). The platform re-imports driver sources whenever the BSP is regenerated, so an unpatched installation reintroduces the error.

To check which compiler configured the BSP, read `platform/.../bsp/libsrc/build_configs/gen_bsp/compile_commands.json`. To force reselection, delete `platform/.../bsp/libsrc/build_configs` and rebuild from a session whose PATH does not contain the other toolchain.

The application runs (RGB LED cycles) but the USB console is silent. `xil_printf` output is directed to the DIP-header UART. The BSP stdout has reverted to `uart_1`: redo Section 7.6, then rebuild both the platform and the application. The compiled setting is recorded in `platform/export/platform/sw/standalone_microblaze_riscv_0/include/bspconfig.h`: `STDOUT_BASEADDRESS` must read `0x40300000` (uart_USB), not `0x40310000` (uart_1).

8 Standalone Boot

This chapter describes how an application is stored in the QSPI flash and started automatically at power-on. A small bootloader (AMD's standard SREC SPI Bootloader, carried inside the hardware image) copies the application from flash into SRAM and starts it, with no PC attached, in under a second after power is applied.

The walkthrough covers the complete path: building the bootloader, merging it into the hardware image, converting an application to a flash image, and programming both into the flash. Everything after the hardware image itself is done in the Vitis Unified IDE.

8.1 How Standalone Boot Works

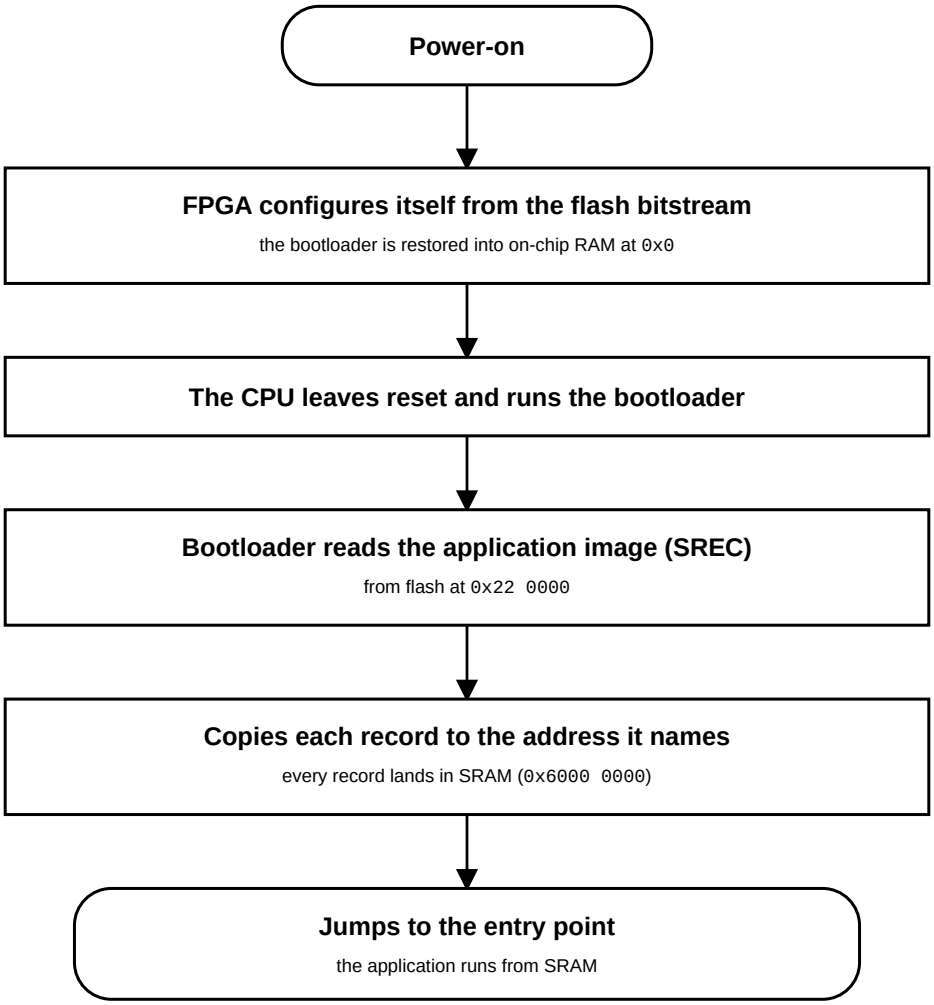


Figure 44: Standalone boot sequence: the FPGA restores the bootloader, which copies the application from flash into SRAM and runs it

The flash holds two independent regions, read by two different readers:

Flash offset	Contents	Read by
`0x000000–0x21FFFF`	Hardware image (bitstream, with the bootloader in its Block RAM initialization)	The FPGA configuration engine, at power-on
`0x220000–0x3FFFFF`	Application slot (SREC image, 1.875 MB)	The bootloader

Block RAM is volatile, so the bootloader cannot live there across a power cycle. Its persistent copy travels inside the bitstream: a bitstream carries the initial contents of every block RAM, and the CPU's local memory is built from block RAM. At every power-on the configuration engine reloads the bitstream from flash, which restores the bootloader. Like a mask ROM, it cannot be corrupted from software. The CPU then leaves reset at address `0x0` and executes it.

The application is stored as an SREC image (Motorola S-record), a text format in which every line carries its own destination address. The bootloader copies each record to the address it names, so the linker script decides where the program runs. The course template links to SRAM at `0x6000_0000`, so the image is loaded there.

Memory	Size	Role
QSPI flash	4 MB	hardware image + bootloader · application slot @ `0x220000` (non-volatile)
SRAM @ `0x60000000`	512 KB	where the application executes (code/data/heap, I/D-cached)
BRAM @ `0x00000000`	128 KB	32 KB bootloader · 32 KB ITCM @ `0x8000` · 64 KB DTCM @ `0x10000`

> **Note:** The repository ships the results of this walkthrough prebuilt in `release/`: `boot_srec.bit` is the hardware image with the bootloader already merged in. To deploy an application with the stock bootloader, skip directly to Section 8.7.

8.2 Prerequisites

- A workspace with the platform and a built application, prepared as described in Section 7
- The hardware image and its memory map from the repository: `release/top.bit` and `release/top_wrapper.mmi`
- Cmod A7-35T board connected via its micro-USB cable
- A serial terminal program for the board's console output

8.3 Create the Bootloader Component

The bootloader is not custom code: it is the stock **SREC SPI Bootloader** example that ships with Vitis, built like any other application component.

8.3.1 New Application from Template

Open the **Examples** view from the left activity bar. Under *Embedded Software Examples*, select **SREC SPI Bootloader** and click **Create Application Component from Template**.

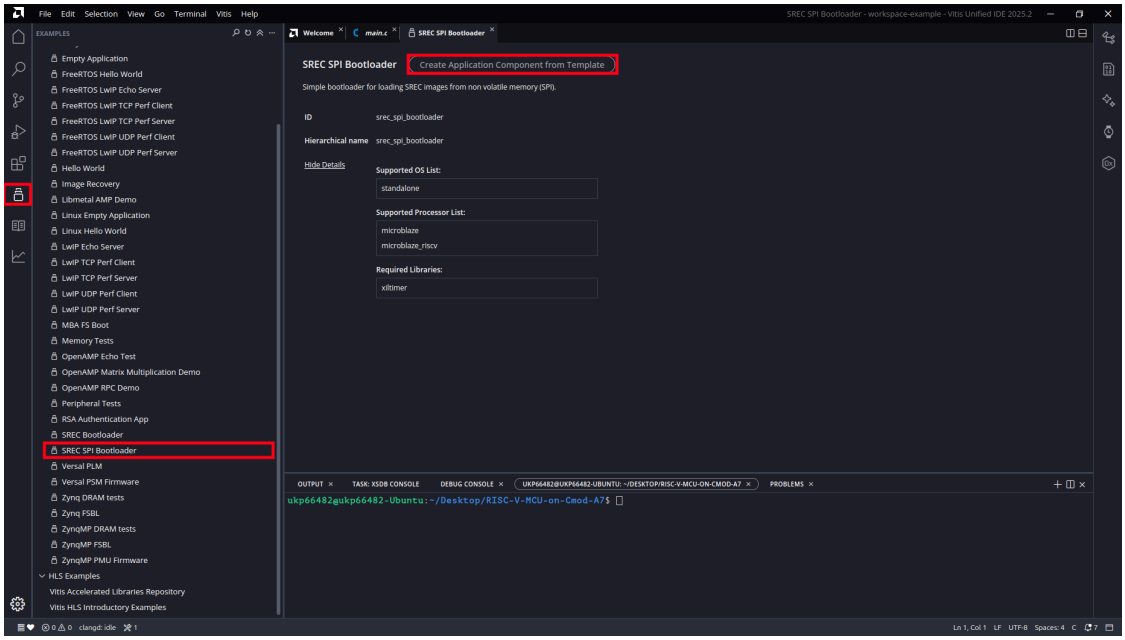


Figure 45: SREC SPI Bootloader Template

8.3.2 Name and Location

Keep the proposed name `srec_spi_bootloader` and the workspace directory as the location. Click **Next**.

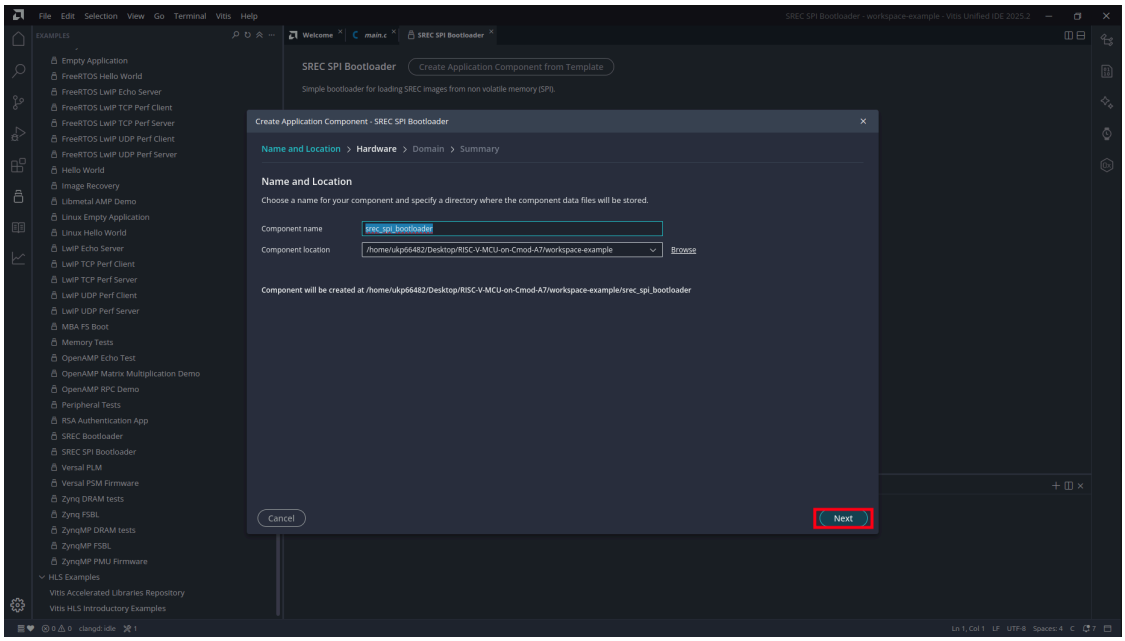


Figure 46: Bootloader Name and Location

8.3.3 Select the Platform

Select the `platform` component (Board: `cmod_a7-35t`). Click **Next**.

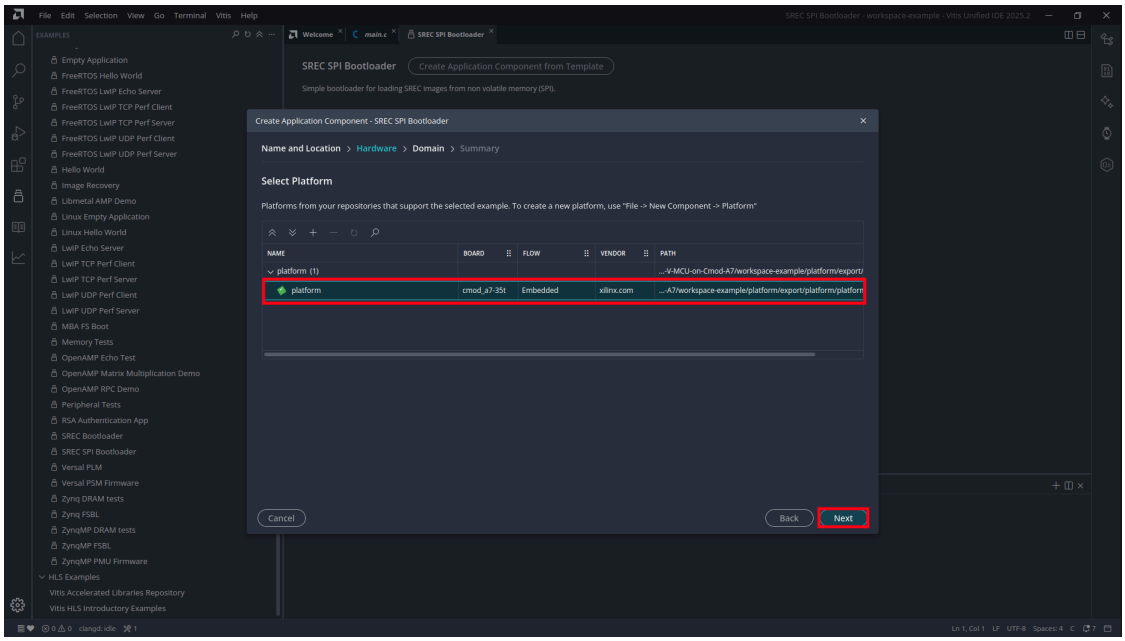


Figure 47: Select Platform

8.3.4 Summary

Review the settings (the domain is `standalone_microblaze_riscv_0`) and click **Finish**.

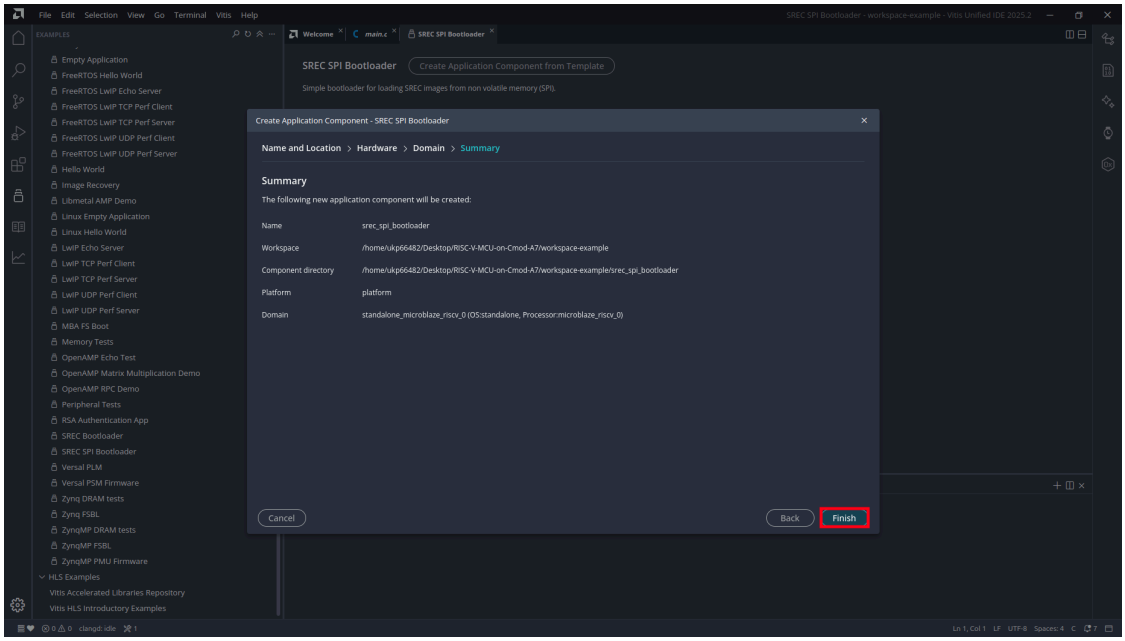


Figure 48: Bootloader Summary

> **Note:** The template automatically selects this board's QSPI flash controller, and it links the bootloader to Block RAM at `0x0`, so the bootloader can copy an application into SRAM without overwriting itself. Neither setting needs to be adjusted.

8.4 Configure the Bootloader

Two source edits configure the template for this board. Both files are under the component's `src/` folder.

8.4.1 Set the Application Slot Address

`blconfig.h` defines where in the flash the bootloader looks for the application. The template ships with a placeholder address and a `#warning` that reminds you to change it:

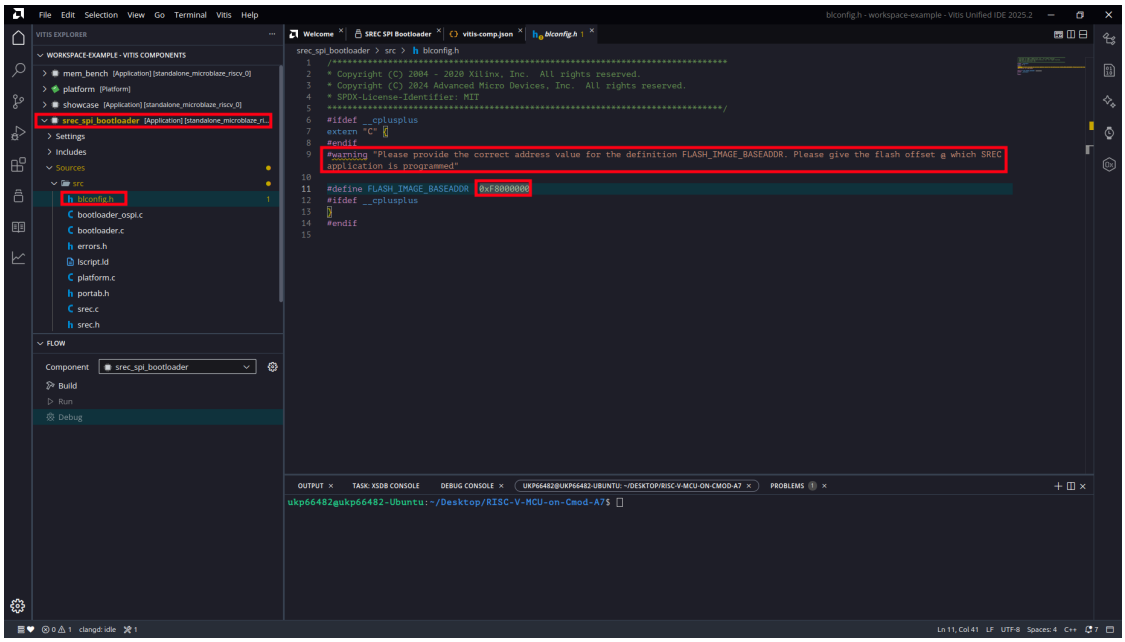


Figure 49: blconfig.h As Generated

Set `FLASH_IMAGE_BASEADDR` to `0x00220000` (the application slot, which starts immediately after the bitstream region) and comment out the `#warning` line:

```
#define FLASH_IMAGE_BASEADDR  0x00220000
```

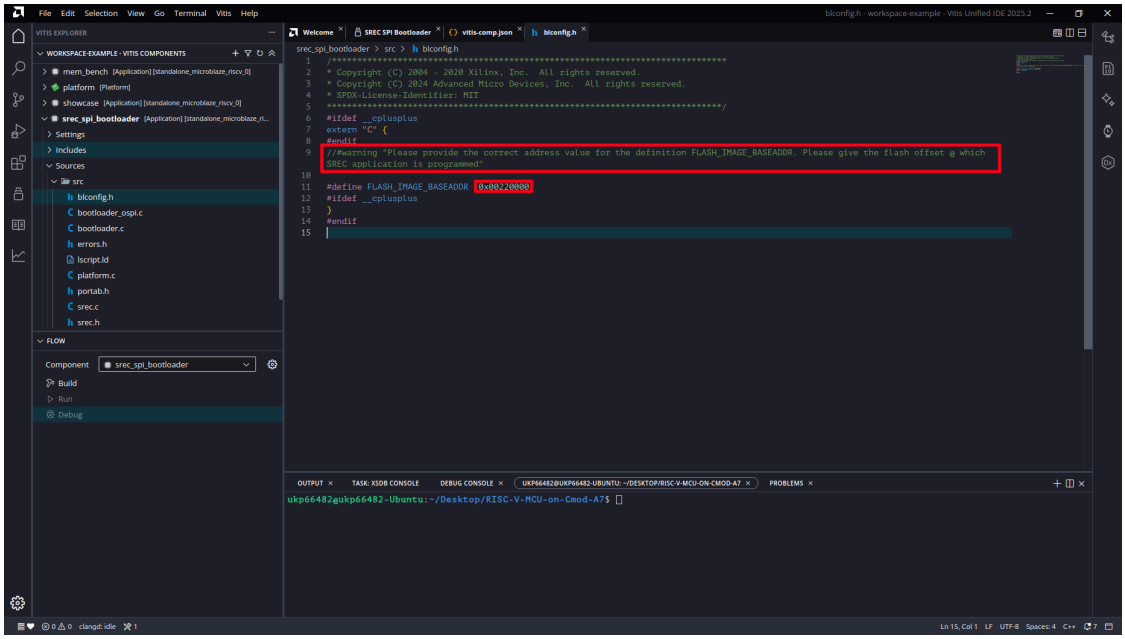


Figure 50: blconfig.h Configured

8.4.2 Disable Verbose Output

`bootloader.c` defines `VERBOSE` by default (line 45):

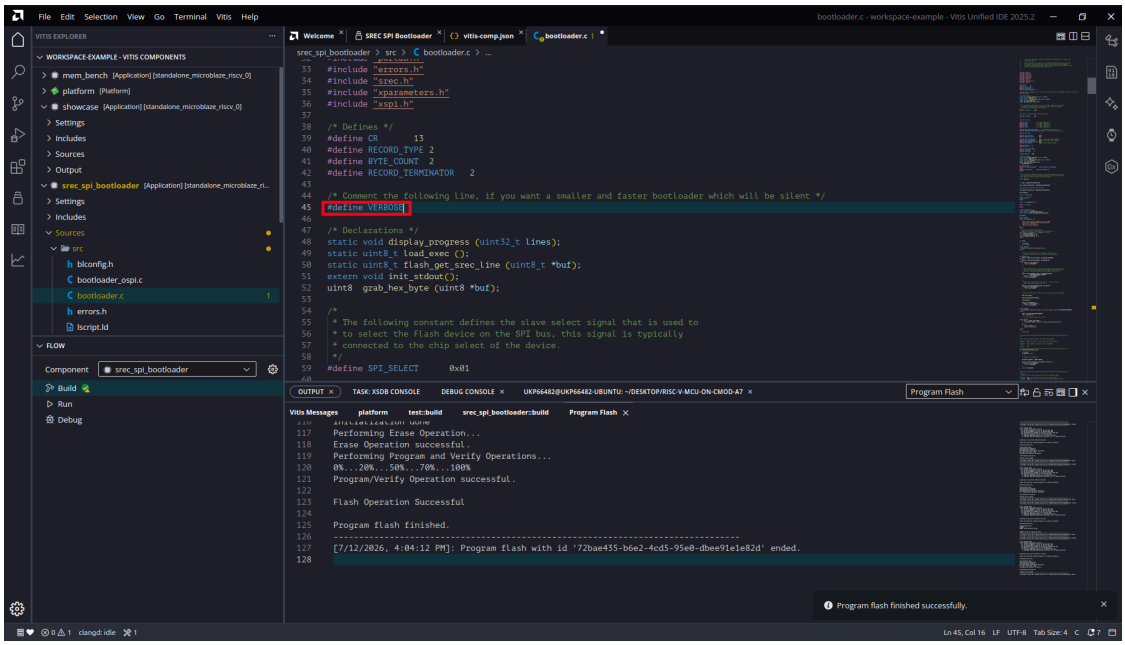


Figure 51: VERBOSE As Generated

Comment it out:

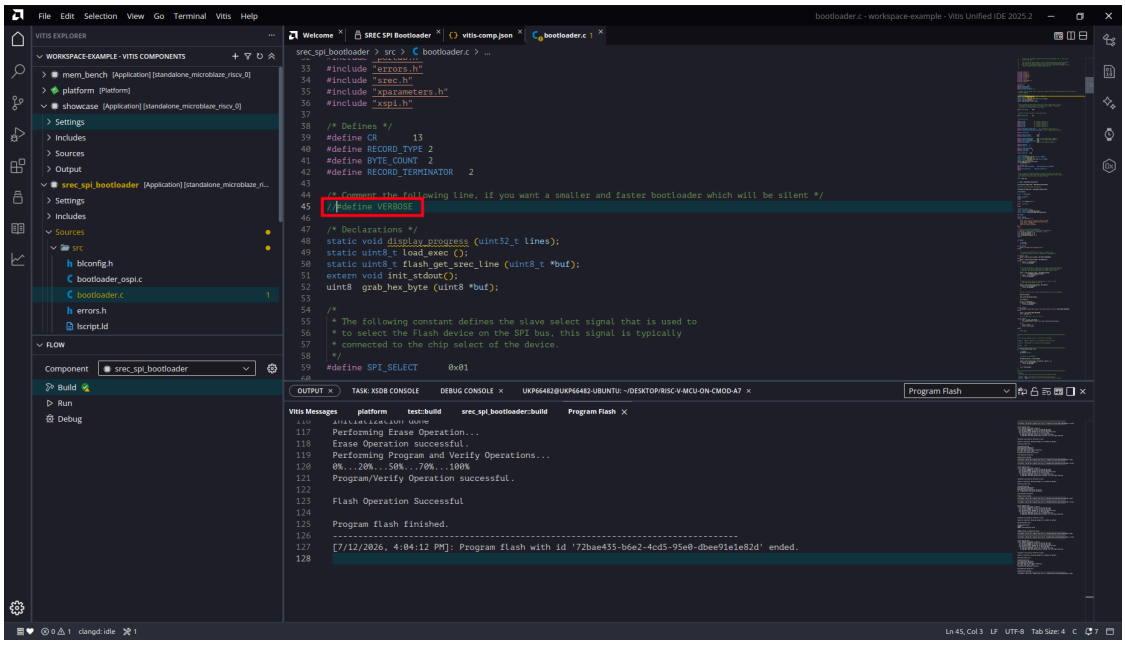


Figure 52: VERBOSE Disabled

> **Note:** With `VERBOSE` enabled the bootloader prints progress for every record it copies, through a UART whose baud divisor it never initializes. The application still boots, but loading takes many seconds instead of well under one second. Disabling `VERBOSE` gives a silent bootloader and restores the sub-second boot. The definition returns whenever the template is regenerated, so re-check this line when creating a new bootloader component.

8.5 Build the Bootloader

In the **FLOW** panel, set the component to `srec_spi_bootloader` and click **Build**.

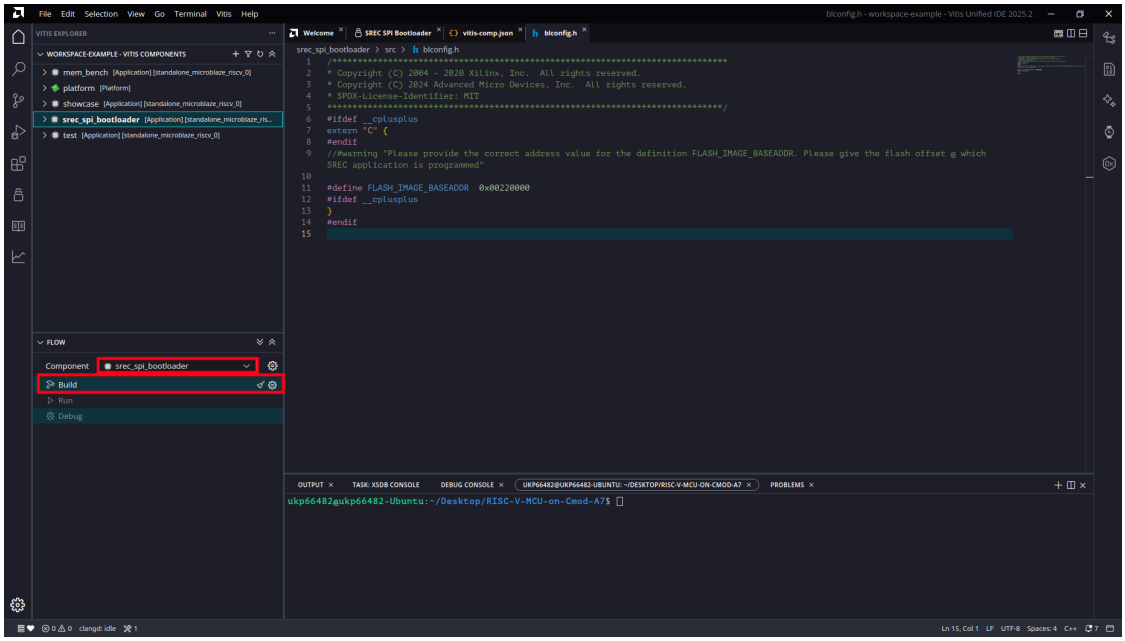


Figure 53: Build the Bootloader

The build produces `srec_spi_bootloader.elf` under the component's `Output` tree (about 16 KB of code, well inside its 32 KB Block RAM region).

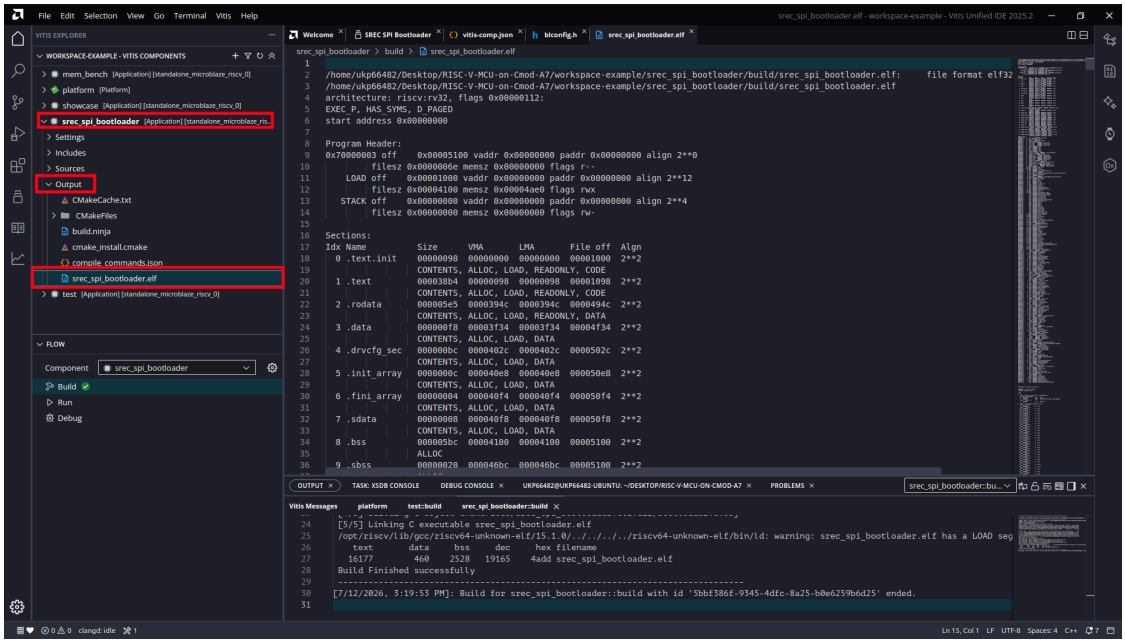


Figure 54: Bootloader ELF Built

8.6 Merge the Bootloader into the Hardware Image

The bootloader ELF is merged into the bitstream's Block RAM initialization with `updatemem`, a command-line tool that ships with Vitis. Only the RAM initialization words change; the logic, routing, and timing of the hardware image are untouched, so the merge takes seconds and needs no hardware rebuild.

Open a terminal in the repository root (**Terminal > New Terminal**) and run:

```
updatemem -force \
  -meminfo release/top_wrapper.mmi \
  -data    workspace-example/srec_spi_bootloader/build/srec_spi_bootloader.elf \
  -bit     release/top.bit \
  -proc    top_i/microblaze_riscv_0 \
  -out     release/boot_srec.bit
```

The inputs and what they mean:

Argument	File	Purpose
`-meminfo`	`top_wrapper.mmi`	Which physical Block RAM cell holds which CPU address (fixed at hardware placement)
`-data`	`srec_spi_bootloader.elf`	The program to place into Block RAM
`-bit`	`top.bit`	The hardware image, Block RAM blank
`-proc`	`top_i/microblaze_riscv_0`	Which processor's memory to initialize
`-out`	`boot_srec.bit`	The bootable hardware image

> **Note:** A successful run prints `updatemem completed successfully` and rewrites the output file. If the tool prints only its usage text, one of the paths is wrong and no output is produced; check the output file's timestamp before using it. The MMI file must come from the same hardware build as the bitstream.

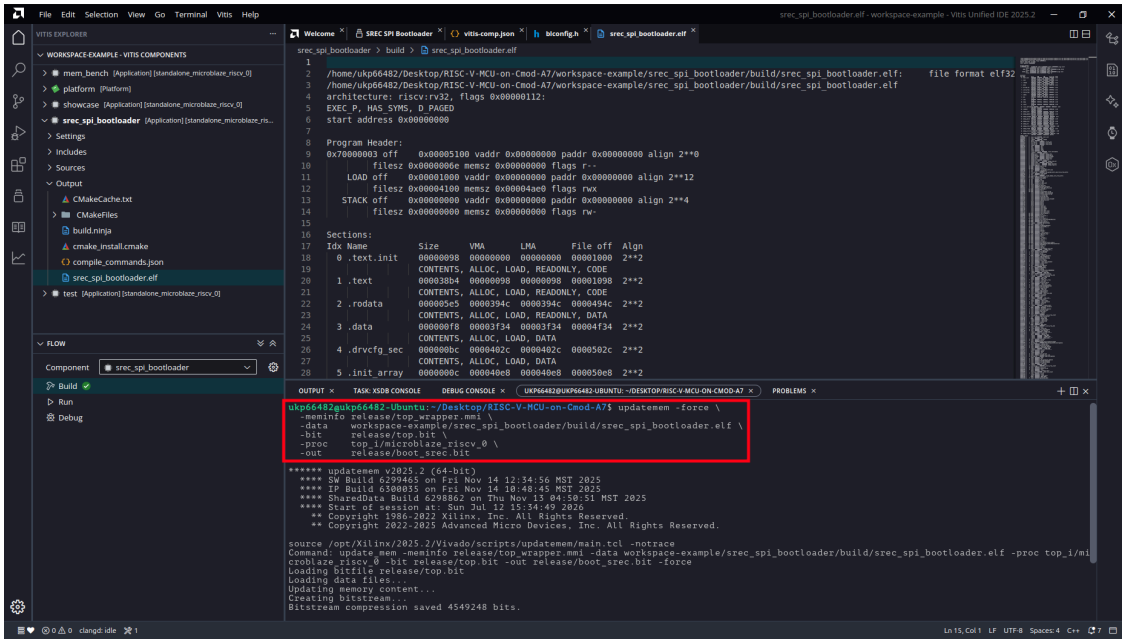


Figure 55: updatemem in the Integrated Terminal

8.7 Prepare the Application Image

The bootloader reads SREC text, so the application ELF is converted with the toolchain's `objcopy`. This walkthrough deploys the `test` application built in Section 7; the same ELF runs unchanged under JTAG and from flash.

```
riscv32-amd-linux-gnu-objcopy -O srec \
  workspace-example/test/build/test.elf \
  workspace-example/test/build/test.srec
```

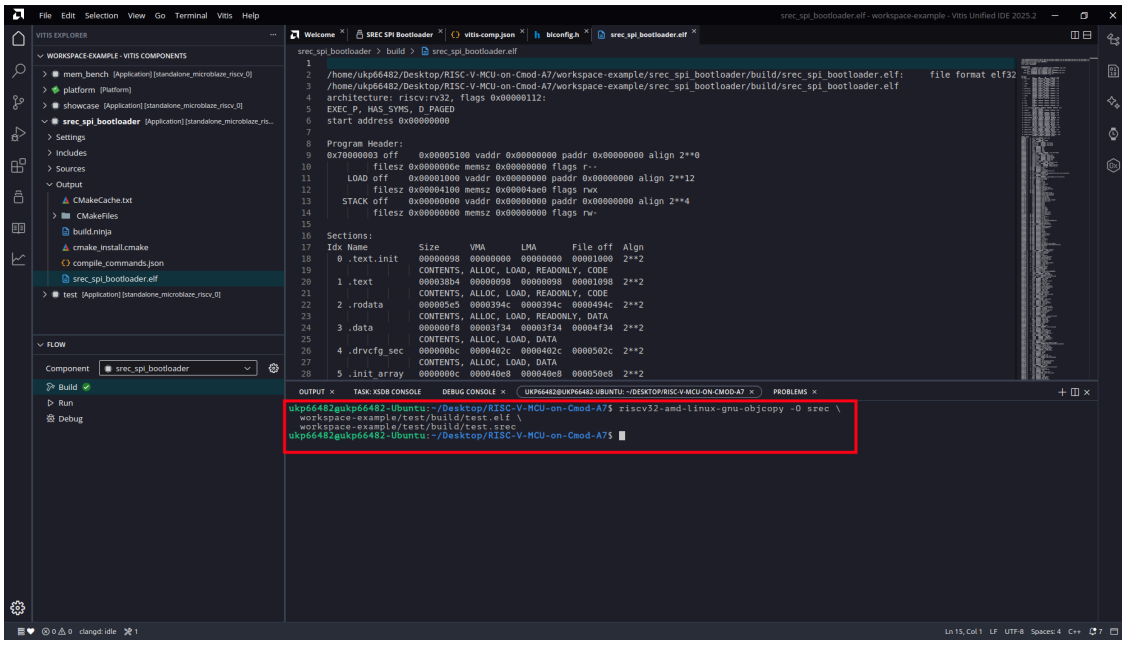


Figure 56: objcopy to SREC

Then give the image a `.bin` extension:

```
cp workspace-example/test/build/test.srec \
   workspace-example/test/build/test_srec.bin
```

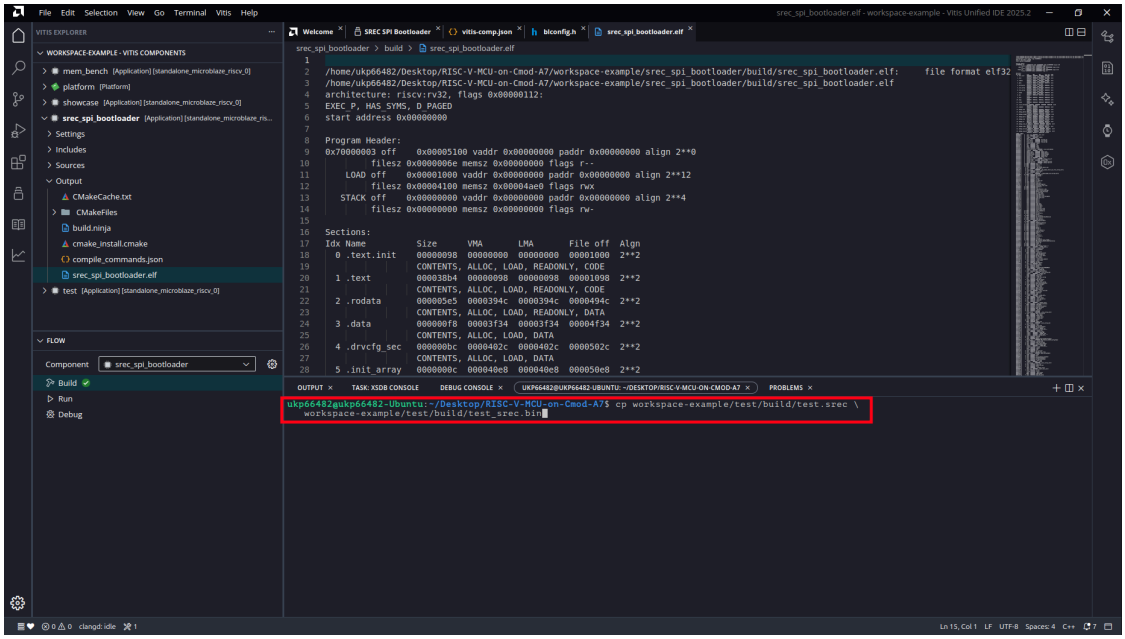


Figure 57: Rename to .bin

> **Note:** The extension matters. The flash programmer treats a `.srec` file as an image format and decodes it to the memory addresses inside it; a `.bin` file is written to the flash byte for byte, which is what the bootloader expects to find in the slot. The file name itself is free; only the `.bin` extension is significant.

Each S3 line of the SREC text embeds its destination address; with the course linker script every record falls in SRAM (addresses beginning `6000`). The image is about three times the size of the binary it encodes; the 1.875 MB slot therefore never limits an application before the 512 KB SRAM does.

8.8 Program the Flash

Both flash regions are written from **Vitis > Program Flash…** in the menu bar. Each write erases and programs only the address range of its own image, so the two regions never disturb each other.

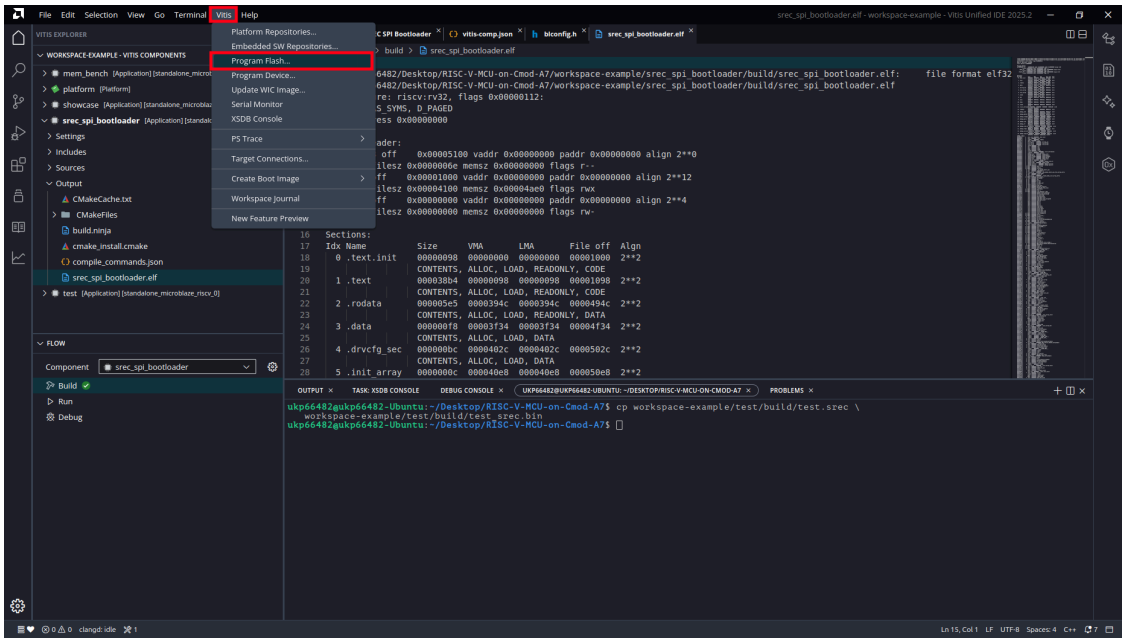


Figure 58: Vitis Program Flash Menu

8.8.1 Program the Hardware Image

This step is needed once, and again only after the bootloader or the hardware design changes. In the *Program Flash Memory* dialog set:

Field	Value
Image File	`release/boot_srec.bit`
Offset	`0x0`
Flash Type	`mx25l3273f-spi-x1_x2_x4`
Verify after flash	checked

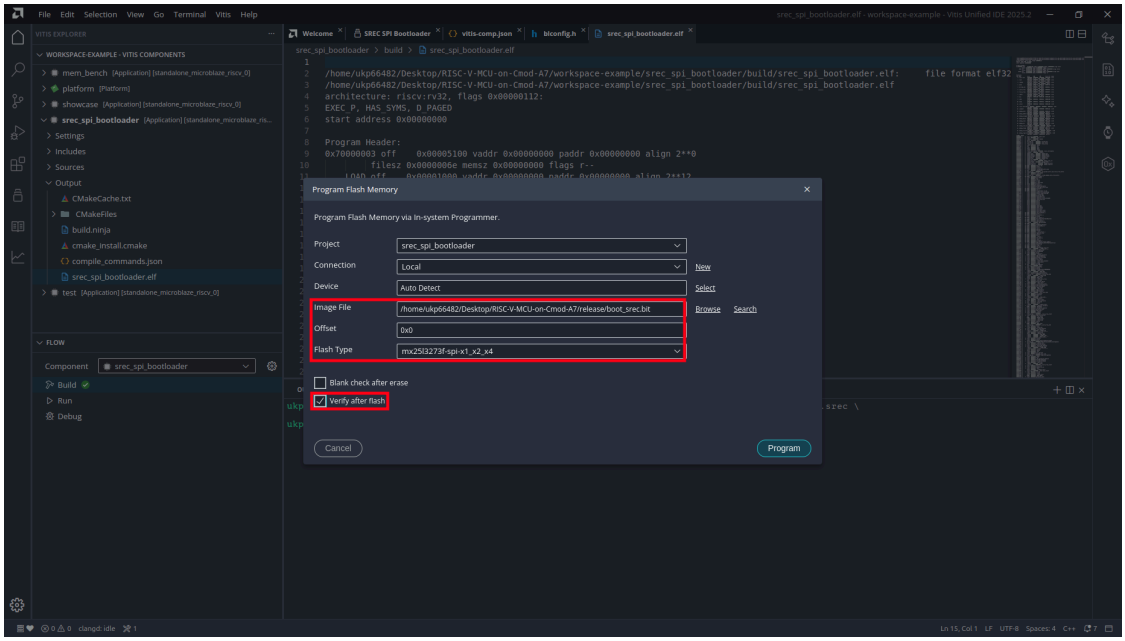


Figure 59: Program the Hardware Image

Click **Program** and wait for *Program flash finished.*

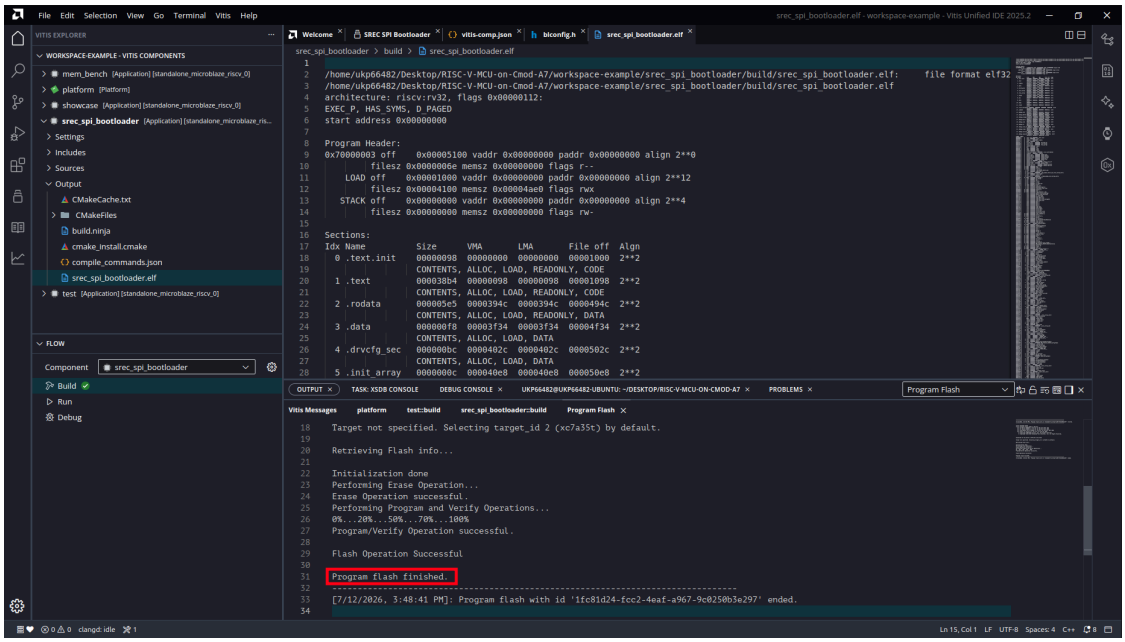


Figure 60: Hardware Image Programmed

> **Note:** The flash type entry covers the board's Macronix MX25L3273F device. Older boards carry a Micron N25Q032A instead; if the IC3 marking says N25Q032, select `n25q32-3.3v-spi-x1_x2_x4`.

8.8.2 Program the Application

Open **Vitis > Program Flash…** again and set:

Field	Value
Image File	the `.bin` application image
Offset	`0x220000`
Flash Type	`mx25l3273f-spi-x1_x2_x4`
Verify after flash	checked

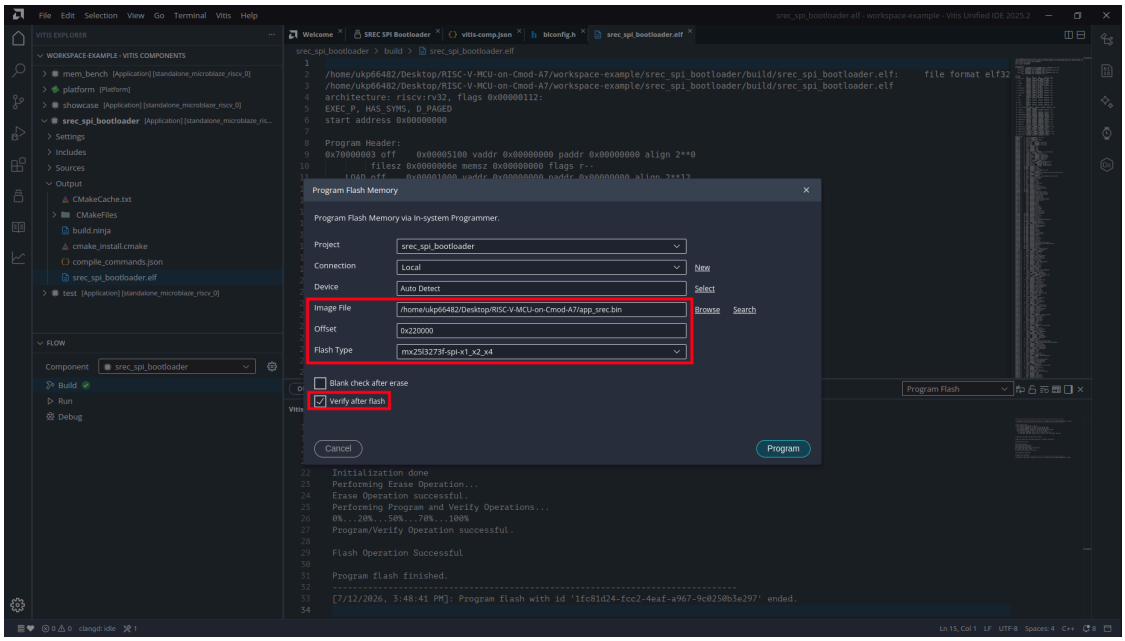


Figure 61: Program the Application Slot

Click **Program** and wait for *Program flash finished.*

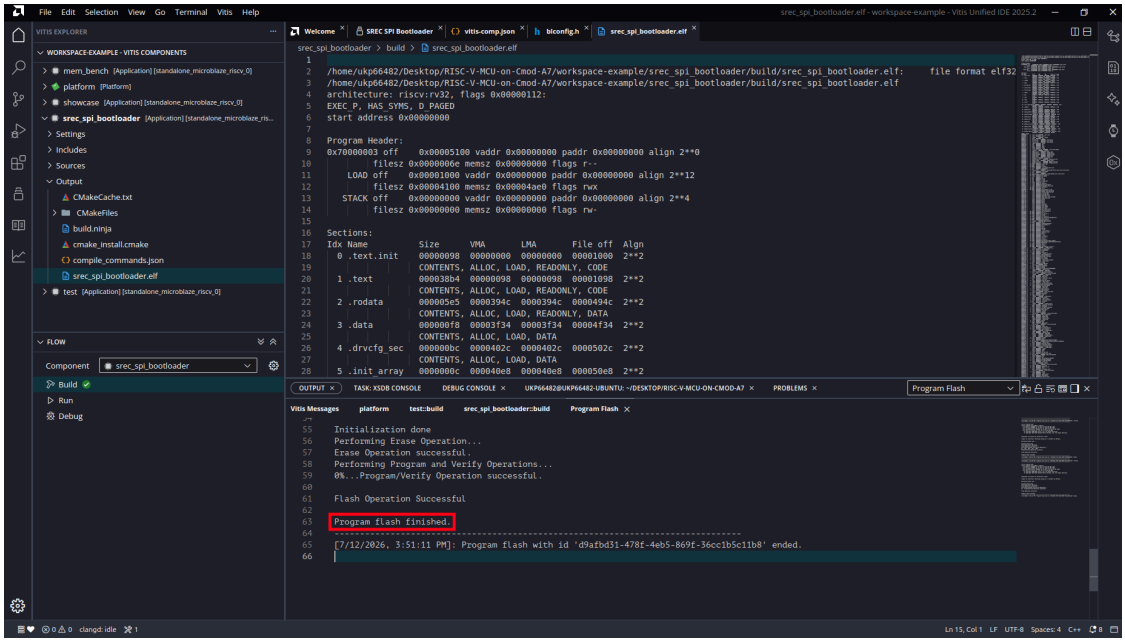


Figure 62: Application Slot Programmed

8.9 Boot and Verify

Disconnect the USB cable and plug it back in. This is a true cold boot: the FPGA configures itself from the flash, the bootloader loads the application, and the application runs, with no debugger and no IDE involved.

Open a serial terminal at 115200 8N1 (the Vitis **Serial Monitor**, GTKTerm, or pyserial's miniterm; the board enumerates as two ports and the UART is usually the higher-numbered one). The course template prints its memory tour, followed by the running status output:

```
RISC-V MCU on Cmod A7 - memory tour
  main()       @ 0x600009EC   (SRAM,  cached)
  blink_step() @ 0x00008000   (ITCM,  1-cycle)
  blink_count  @ 0x00010000   (DTCM,  1-cycle)
  stack        @ 0x0001FFC0   (DTCM,  grows down)
```

> **Note:** The reset button (BTN0) restarts the CPU into the bootloader, which re-copies the application from flash, so a reset behaves like a power-cycle. Modified `.data` globals are restored because the image is re-read from flash at every boot.

8.10 Updating the Application

After the one-time setup above, the day-to-day deployment loop has three steps, none of which touch the bootloader or the hardware image:

1. Build the application in Vitis.
2. Convert and rename the image (the two commands in the application image step).
3. **Vitis > Program Flash…** to offset `0x220000`.

Power-cycle, and the new application boots.

8.11 Working with JTAG Debug Mode

Both modes coexist on the same board with no switching:

	JTAG (Vitis Run/Debug)	Standalone (flash)
Code goes to	RAM directly (volatile)	Flash (persistent)
After power-cycle	the flashed application boots	the deployed application boots
Breakpoints / stepping	yes	no
Typical use	daily development loop	shipping a finished program

The typical development cycle is edit, build, and JTAG Run/Debug. When the program works, deploy it to flash once. JTAG always takes priority over the bootloader: it halts the CPU before the bootloader runs, so the flash contents cannot interfere with a debug session.

8.12 Troubleshooting

1. **Boot works but takes many seconds:** `VERBOSE` is still defined in `bootloader.c`. Comment it out, rebuild the bootloader, re-run `updatemem`, and reprogram the hardware image.
2. **Nothing at all after power-on (no LED, no output):** The flash holds `top.bit` (blank Block RAM, no bootloader) instead of `boot_srec.bit`, or the hardware image region is empty or corrupt. Reprogram the hardware image. JTAG remains available for recovery.
3. **`updatemem` printed only usage text:** One of the argument paths is wrong; no output file was written, and reprogramming would flash the previous image. Fix the path and confirm the `updatemem completed successfully` message.
4. **Program Flash fails with `Flash Operation Failed`:** Another process is holding the JTAG connection, typically a Vitis debug session or an XSDB console. Close it (or unplug and replug the board) and retry.
5. **Garbage on the terminal:** The baud rate must be 115200. The course template sets the UART divisor in `mcu_init()`; the plain Hello World template does not set it at all.
6. **Deployed, but silent after power-on (works under JTAG):** The ELF was not linked with the course template's `lscript.ld`. The bootloader honors the SREC record addresses, and a default all-BRAM layout collides with the bootloader itself. Rebuild with the template and redeploy.
7. **Board boots an old application:** The flash write did not complete. Repeat the application programming step and confirm the verify pass succeeds.